



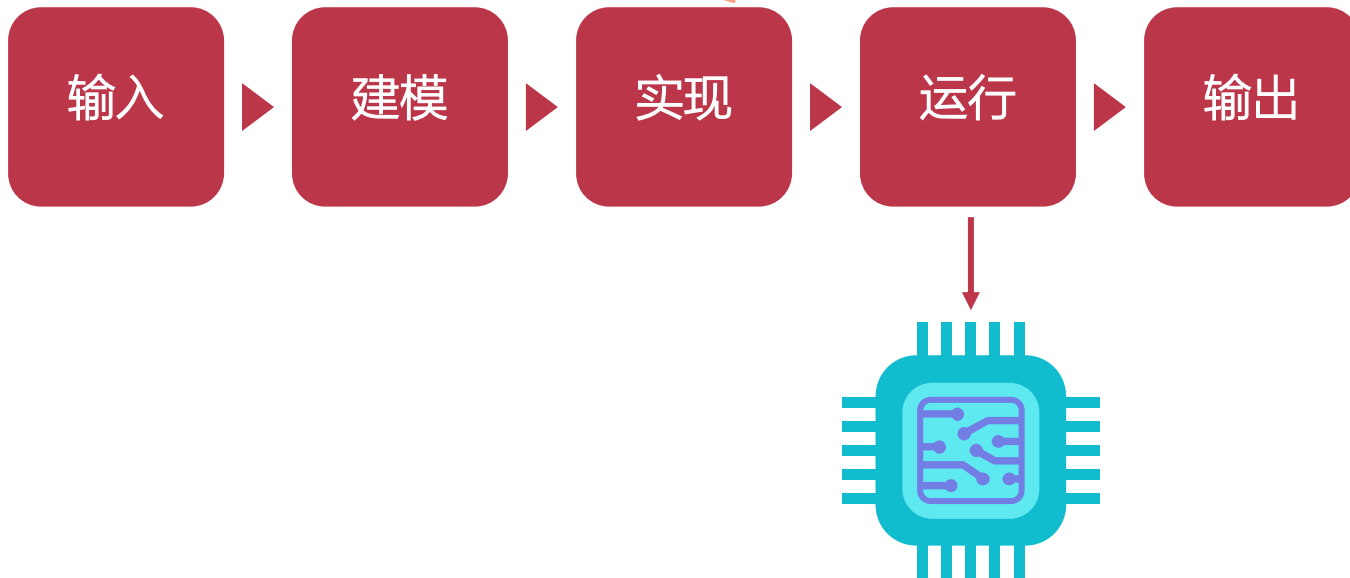
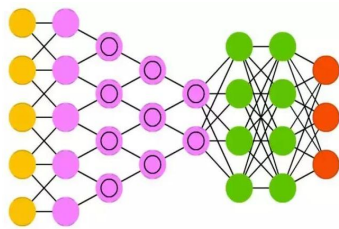
智能计算系统

第六章 面向深度学习的 处理器原理

北京邮电大学 人工智能学院

何召锋、徐振博、项刘宇

运行



智能计算系统中的处理器

用来处理智能任务的处理器

- ▶ 可以是CPU、GPU、FPGA等
- ▶ 也可以是专用的深度学习处理器（DLP）

智能计算系统中的处理器

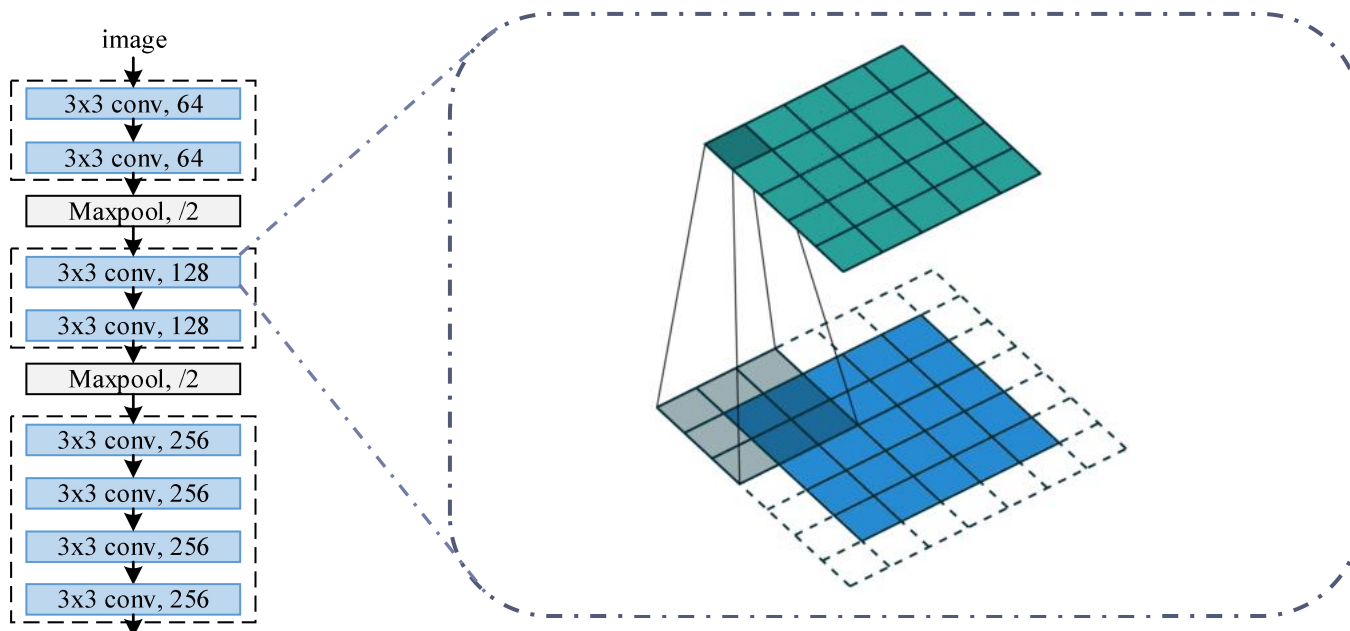
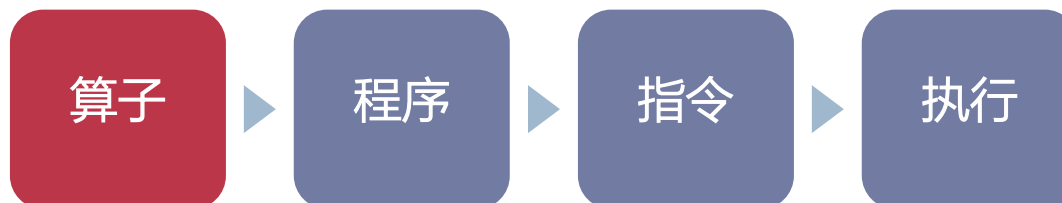
本节课将介绍以下几种类型：

- ▶ 通用处理器：如CPU，以标量运算作为基本运算。
- ▶ 向量处理器：如GPU，以向量运算作为基本运算。
- ▶ 深度学习处理器：以矩阵运算等作为基本运算。

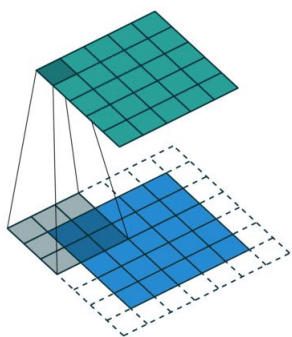
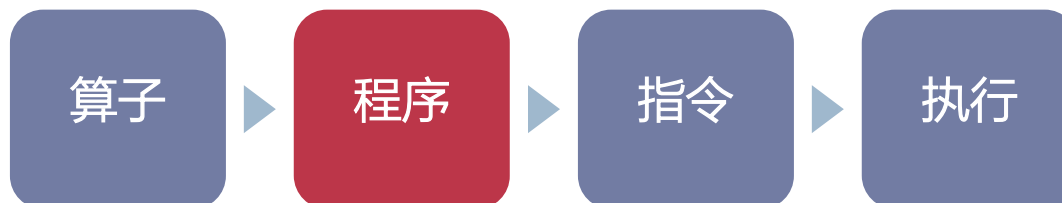
前置知识

- ▶ 计算机体系结构
- ▶ 处理器的构成
- ▶ 处理器的工作流程
- ▶ 处理器的存储

代码的执行过程

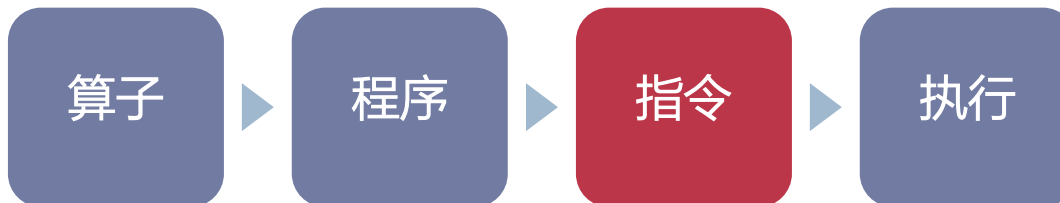


代码的执行过程



```
for(co=0; co<Co; co++)  
  for(yo=0; yo<Ho; yo++)  
    for(xo=0; xo<Wo; xo++) {  
      output[co][yo][xo] = bias[co];  
      for (yk=0; yk<Hk; yk++)  
        for (xk=0; xk<Wk; xk++)  
          for (ci=0; ci<Ci; ci++) {  
            output[co][yo][xo] += weight[co][ci][yk][xk] *  
              input[ci][yo*stride+yk][xo*stride+xk];  
          }  
    }  
}
```

代码的执行过程



```
for(co=0; co<Co; co++)  
for(yo=0; yo<Ho; yo++)  
for(xo=0; xo<Wo; xo++) {
```

```
    output[co][yo][xo] = bias[co];
```

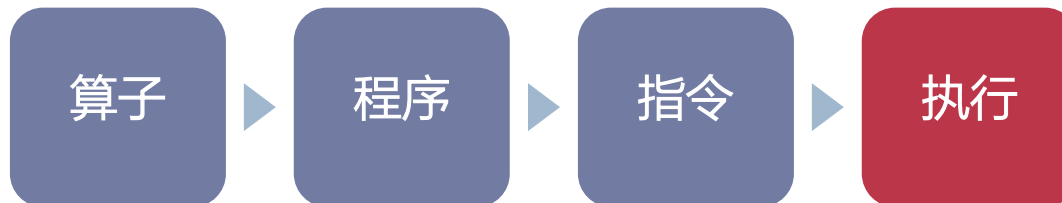
```
    for (yk=0; yk<Hk; yk++)  
    for (xk=0; xk<Wk; xk++)  
    for (ci=0; ci<Ci; ci++) {
```

```
        output[co][yo][xo] +=  
            weight[co][ci][yk][xk] *  
            input[ci]  
                [yo*stride+yk]  
                [xo*stride+xk];
```

```
    }  
}
```

```
.L2:    cmp     DWORD PTR [rsp-20], eax  
        jle     .L1  
        imul    rdx, rax, 27  
        mov     QWORD PTR [rsp-40], r10  
        xor     r9d, r9d  
        mov     QWORD PTR [rsp-32], rdx  
        xor     edx, edx  
        mov     DWORD PTR [rsp-24], edx  
        mov     edi, DWORD PTR [rsp-16]  
        cmp     DWORD PTR [rsp-24], edi  
        jge     .L16  
        mov     rdx, QWORD PTR [rsp-40]  
        xor     ebx, ebx  
        xor     r11d, r11d  
        mov     ecx, DWORD PTR [rsp-12]  
        cmp     r11d, ecx  
        jge     .L17  
        movss   xmm0, DWORD PTR bias[0+rax*4]  
        mov     r12, QWORD PTR [rsp-32]  
        xor     r8d, r8d  
        movss   DWORD PTR [rdx], xmm0  
        mov     edi, DWORD PTR [rsp-8]  
        cmp     r8d, edi  
        jge     .L7  
        lea     ecx, [r8+r9]  
        lea     rbp, weight[0+r12*4]  
        xor     edi, edi  
        movsx   rcx, ecx  
        imul    rcx, rcx, 224  
        mov     r14d, DWORD PTR [rsp-4]  
        cmp     edi, r14d  
        jge     .L5  
        lea     r13d, [rdi+rbx]  
        mov     r14, rbp  
        mov     r13, r13d  
        add     r13, rcx  
        lea     r15, input[0+r13*4]
```


代码的执行过程



```

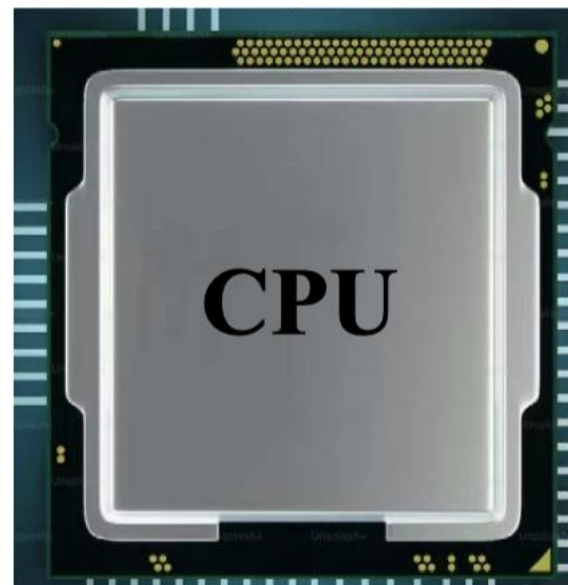
.L2:  cmp    DWORD PTR [rsp-20], eax
     jle     .L1
     imul   rdx, rax, 27
     mov    QWORD PTR [rsp-40], r10
     xor    r9d, r9d
     mov    QWORD PTR [rsp-32], rdx
     xor    edx, edx
     mov    DWORD PTR [rsp-24], edx
.L12: mov    edi, DWORD PTR [rsp-16]
     cmp    DWORD PTR [rsp-24], edi
     jge     .L16
     mov    rdx, QWORD PTR [rsp-40]
     xor    ebx, ebx
     xor    r11d, r11d
.L10:  mov    ecx, DWORD PTR [rsp-12]
     cmp    r11d, ecx
     jge     .L17
     movss  xmm0, DWORD PTR bias[0+rax*4]
     mov    r12, QWORD PTR [rsp-32]
     xor    r8d, r8d
     movss  DWORD PTR [rdx], xmm0
.L13:  mov    edi, DWORD PTR [rsp-8]
     cmp    r8d, edi
     jge     .L7
     lea    ecx, [r0+r9]
     lea    rbp, weight[0+r12*4]
     xor    edi, edi
     movsx  rcx, ecx
     imul   r12, rcx, 224
.L18:  mov    r14d, DWORD PTR [rsp-4]
     cmp    edi, r14d
     jge     .L5
     lea    r13d, [rdi+rbx]
  
```

```

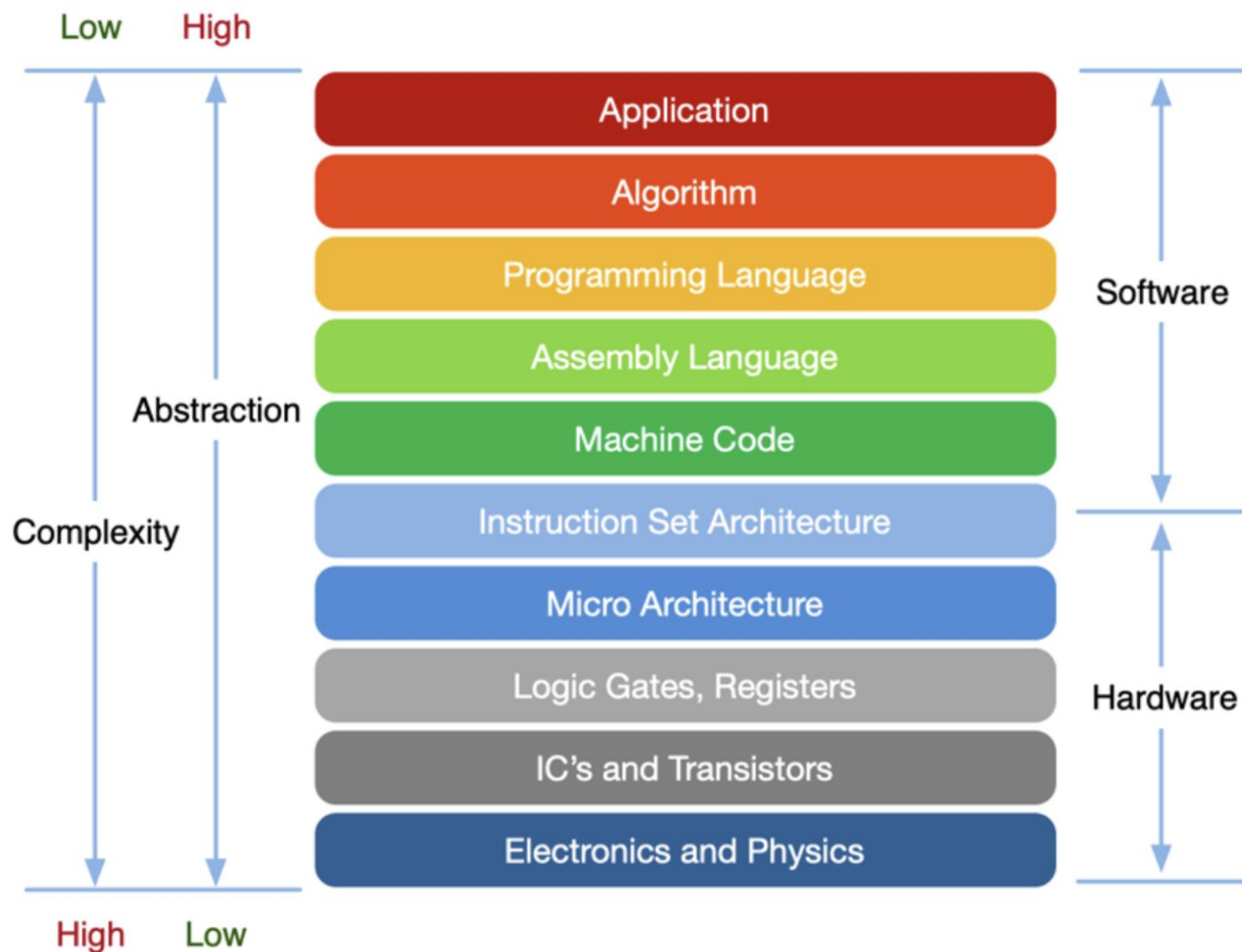
     mov    r14, rbp
     movsx  r13, r13d
     add    r13, rcx
     lea    r15, input[0+r13*4]
     xor    r13d, r13d
     cmp    r13d, DWORD PTR [rsp+56]
     jge     .L18
     movss  xmm0, DWORD PTR [r14]
     mulss  xmm0, DWORD PTR [r15]
     inc    r13d
     add    r14, 36
     addss  xmm0, DWORD PTR [rdx]
     add    r15, 200704
     movss  DWORD PTR [rdx], xmm0
     inc    edi
     add    rbp, 4
     jmp    .L8
     inc    r8d
     add    r12, 3
     jmp    .L3
     inc    r11d
     add    rdx, 4
     add    ebx, esi
     jmp    .L10
     inc    QWORD PTR [rsp-24]
     add    r9d, esi
     mov     QWORD PTR [rsp-40], r88
     inc    rax
     add    r10, 197136
     jmp    .L2
.L1:
.L16:
.L17:
.L19:
  
```

```

64 39 44 24 EC 0F 8E 00 01 00 00 48 6B D0 1B 4C
89 54 24 D8 45 31 C9 48 89 54 24 E0 31 D2 89 54
24 E8 8B 7C 24 F0 39 7C 24 E8 0F 8D CC 00 00 00
48 8B 54 24 D8 31 DB 45 31 DB 8B 4C 24 F4 41 39
CB 0F 8D A0 00 00 00 F3 0F 10 04 85 00 00 00 00
4C 8B 64 24 E0 45 31 C0 F3 0F 11 02 8B 7C 24 F8
41 39 F8 7D 74 43 8D 0C 08 4A 8D 2C A5 00 00 00
00 31 FF 48 63 C9 48 69 C9 E0 00 00 00 44 8B 74
24 FC 44 39 F7 7D 49 44 8D 2C 1F 49 89 EE 4D 63
ED 49 01 CD 4E 8D 3C AD 00 00 00 00 45 31 ED 44
3B 6C 24 38 7D 22 F3 41 0F 10 06 F3 41 0F 59 07
41 FF C5 49 83 C6 24 F3 0F 58 02 49 81 C7 00 10
03 00 F3 0F 11 02 EB D7 FF C7 48 83 C5 04 EB AD
41 FF C0 49 83 C4 03 EB 83 41 FF C3 48 83 C2 04
01 F3 E9 53 FF FF FF FF 44 24 E8 41 01 F1 48 81
44 24 D8 78 03 00 00 E9 26 FF FF FF 48 FF C0 49
81 C2 10 02 03 00 E9 F6 FE FF FF
  
```



计算机体系结构



指令集

```
1  #include "stdio.h"
2
3  int main()
4  {
5      int sum=0;
6      for(int i=1;i<=100;i++)
7      {
8          sum+=i;
9      }
10     printf("sum=%d\n",sum);
11     return 0;
12 }
```

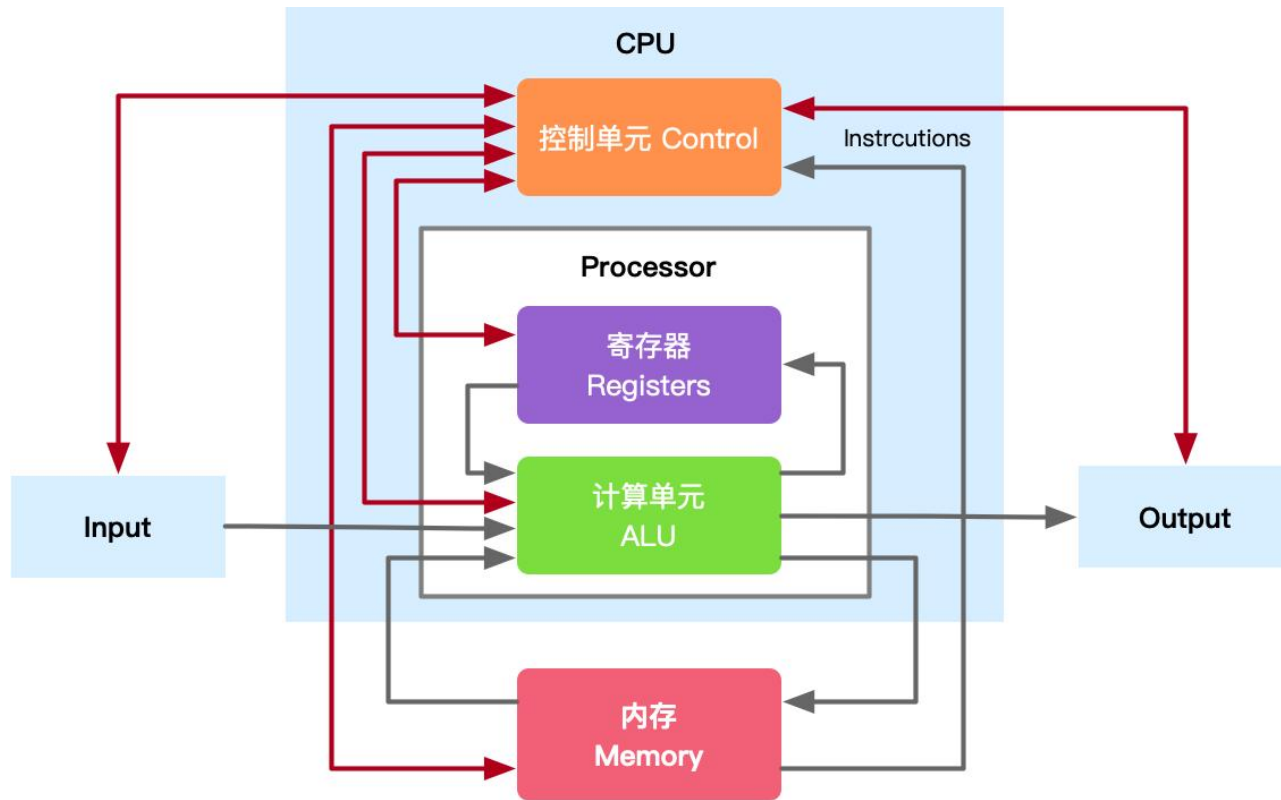
C语言

```
7      //相当于int sum=0;
8      mov ebx,0
9      //相当于 int i=1
10     mov eax,1
11
12     //相当于i<=100
13 ee:   cmp eax,100
14       jg end
15     //相当于 sum+=i
16     add ebx,eax
17     //相当于 i++
18     inc eax
19     jmp ee
20
21     //相当于 printf
22 end:  push ebx
23       push str
24       call printf
```

汇编

处理器的构成：CPU v.s. GPU

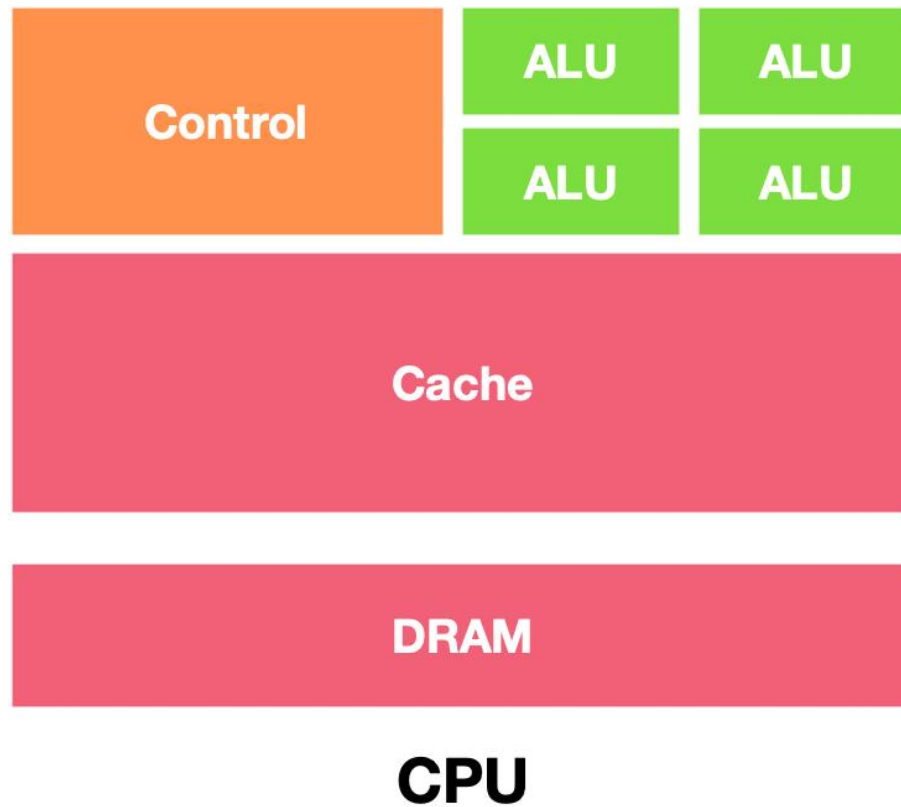
- ▶ CPU构成：运算器、控制器和寄存器三大部分。



<https://github.com/chenzomi12/DeepLearningSystem/>

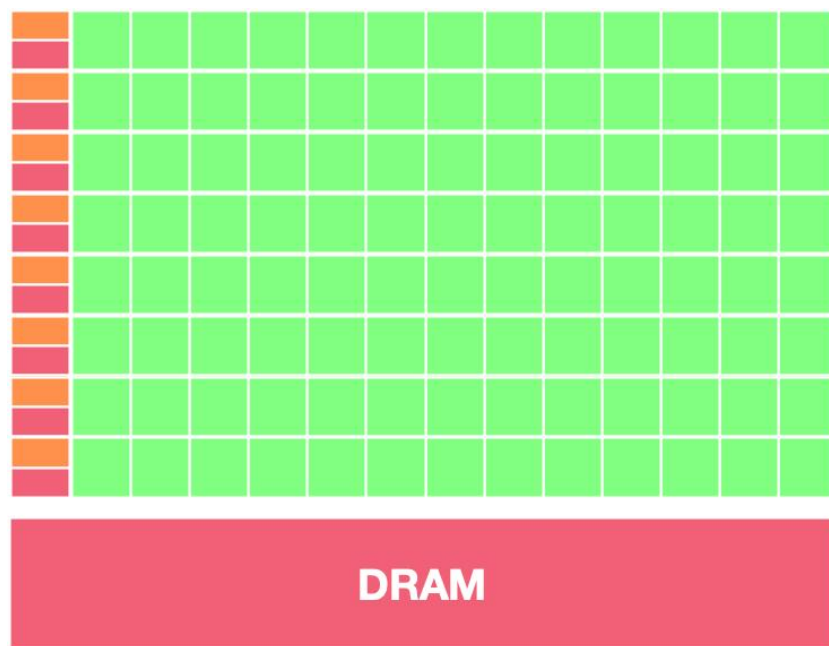
处理器的构成：CPU v.s. GPU

- ▶ CPU架构：擅长逻辑控制而不擅长计算



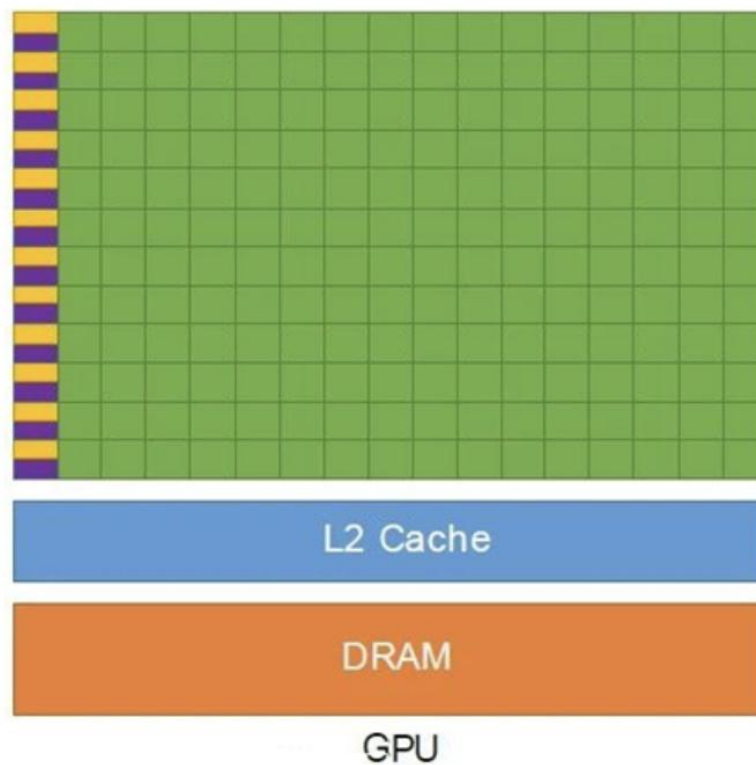
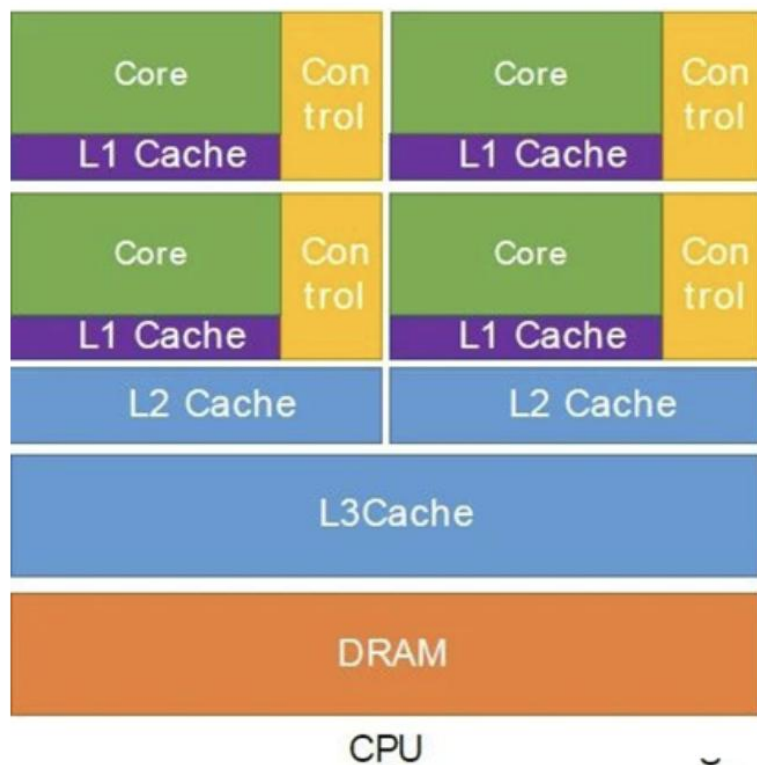
处理器的构成：CPU v.s. GPU

- ▶ GPU的初衷主要是为了接替CPU进行图形渲染的工作，每秒需要处理上亿像素点。考虑到像素点的处理方式大抵相同，因此考虑并行计算。
- ▶ 特点：
 - ▶ 缓存小
 - ▶ 控制简单
 - ▶ 大量ALU计算单元



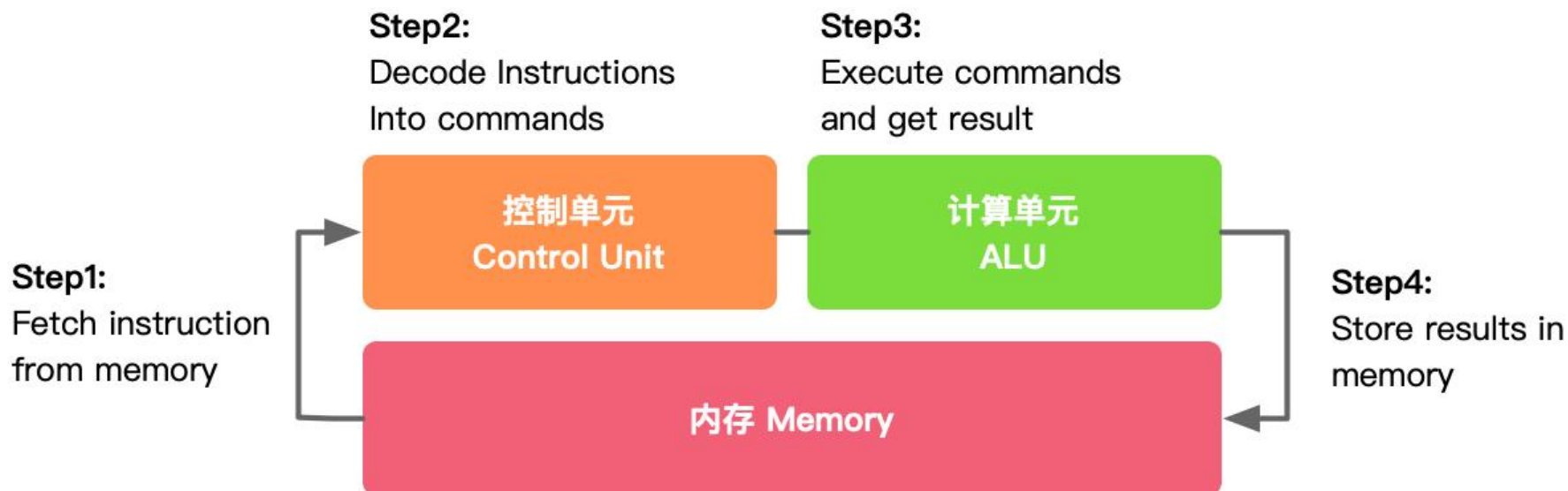
处理器的构成：CPU v.s. GPU

- ▶ GPU几乎主要由计算单元ALU组成，仅有少量的控制单元和存储单元。
- ▶ CPU不仅被Cache占据了大量空间，而且还有复杂的控制逻辑，相比之下计算能力只是CPU很小的一部分。



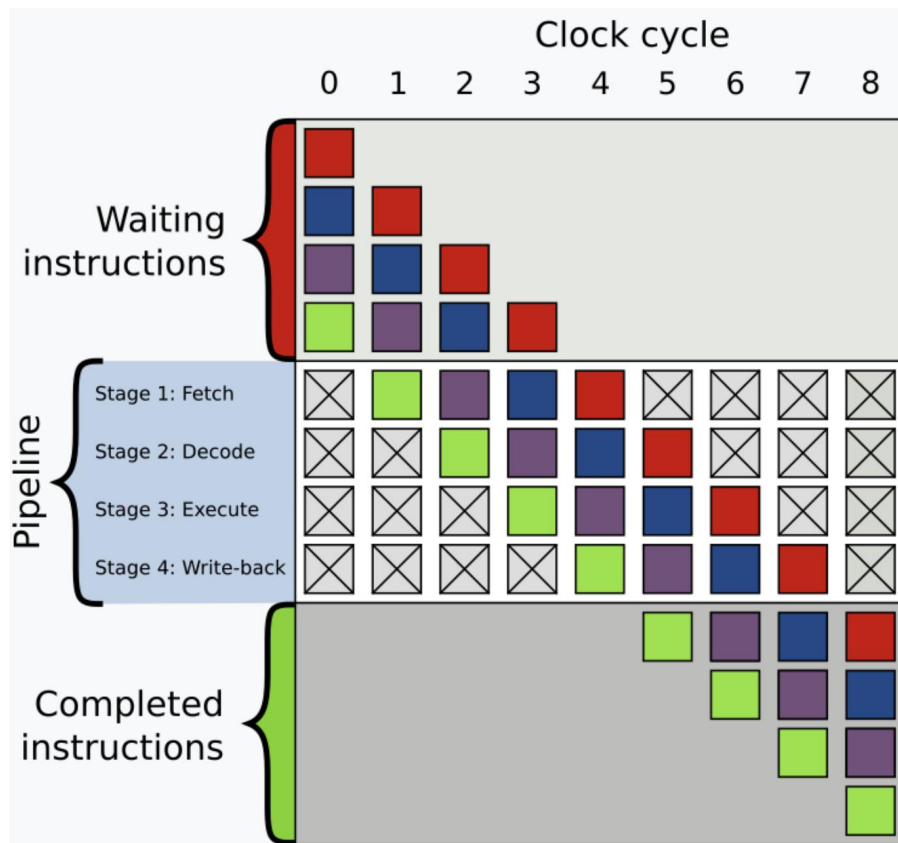
处理器的工作流程

- ▶ 最基本的4级流水线为例：
- ▶ 1. 取指 2. 解码 3. 执行 4. 写回

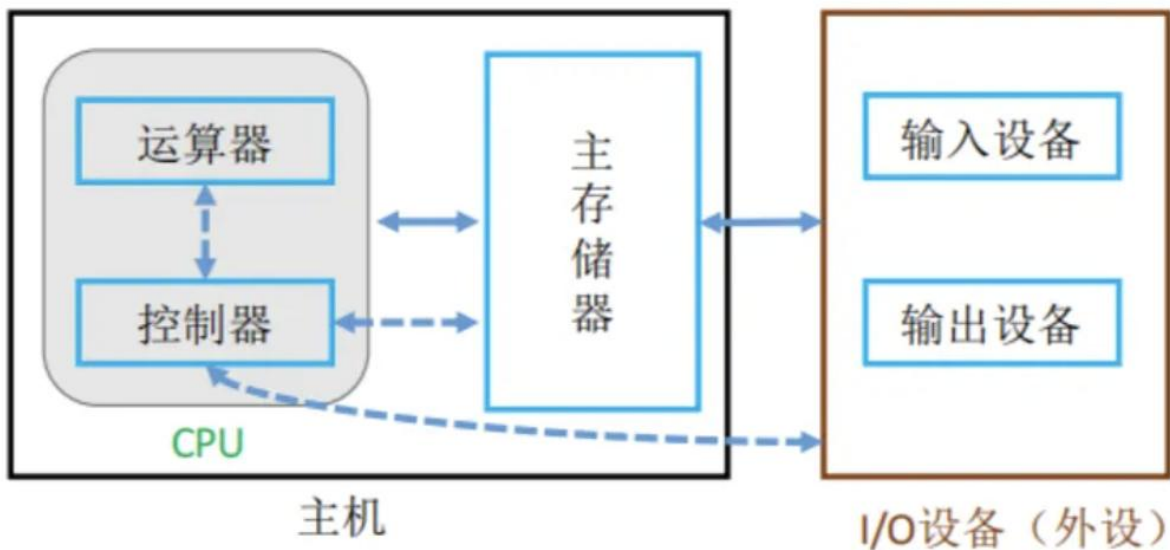


处理器的工作流程

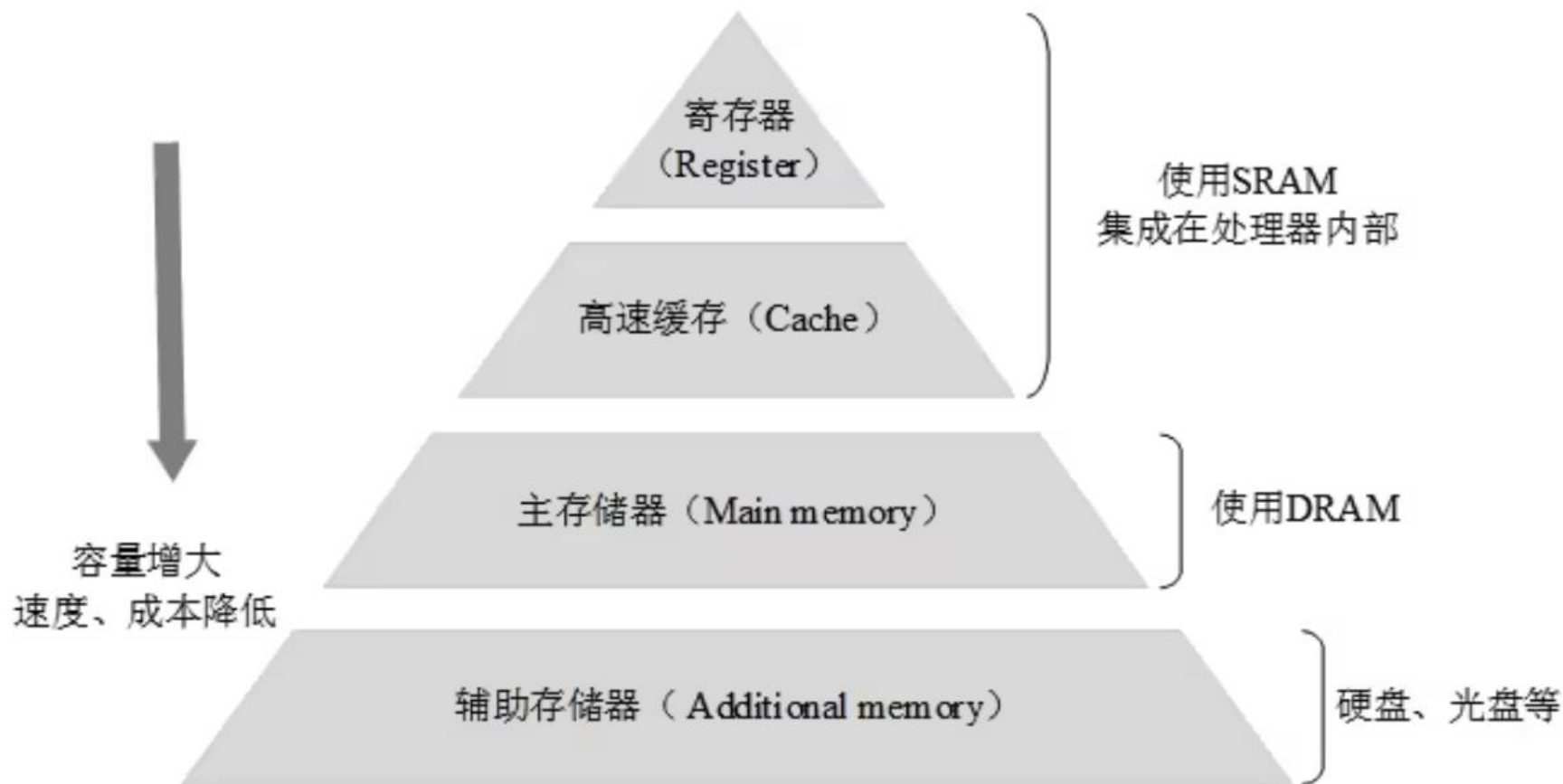
- ▶ 最基本的4级流水线为例：
- ▶ 1. 取指 2. 解码 3. 执行 4. 写回



处理器的存储



处理器的存储



<https://foxsen.github.io/archbase/index.html>

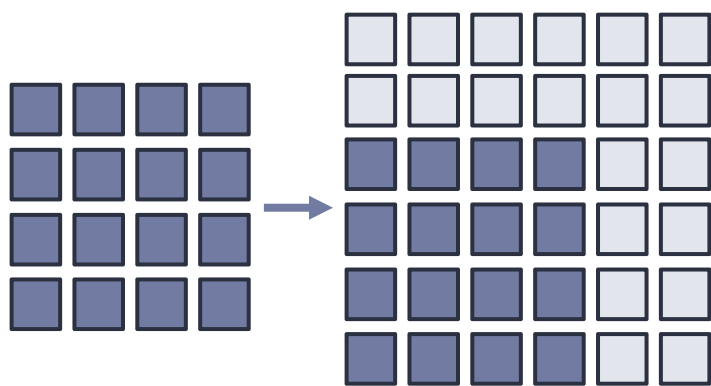
计算机体系结构基础第3版 胡伟武 等

处理器的存储

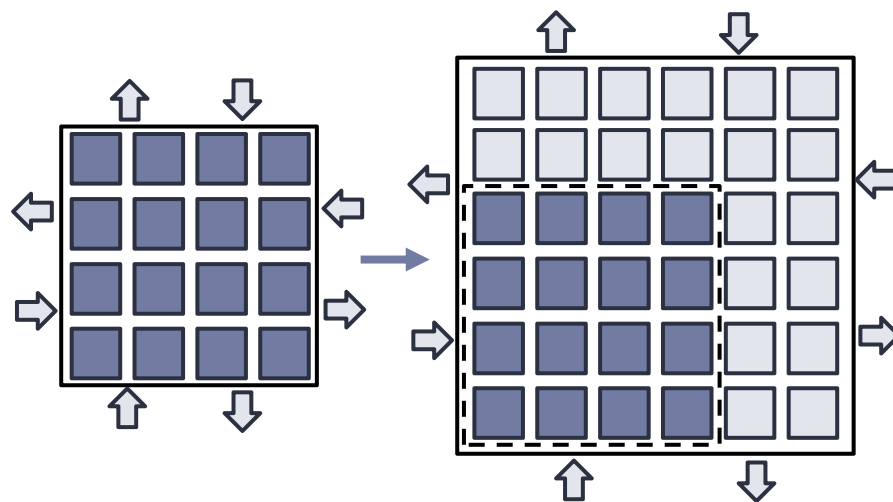


运算与访存

- ▶ 除了处理器的计算能力，分析访存特性也是十分重要的！
 - ▶ 芯片里能存放的**运算器数量**和**芯片面积**成比例增长
 - ▶ 芯片的**访存带宽**是和**芯片周长**成比例增长
 - ▶ 随着计算能力的增加，访存会成为新的瓶颈。



计算能力 $\propto$ 面积



访存能力 $\propto$ 周长

提纲

- ▶ 通用处理器
- ▶ 向量处理器
- ▶ 深度学习处理器
- ▶ 大规模深度学习处理器
- ▶ 总结

通用处理器

为什么选用通用处理器？

- ▶ 普及：每台计算设备都包含CPU
- ▶ 廉价：1分钱获得完备的处理能力
- ▶ 灵活：可与其他任何任务共享计算硬件

适用小模型、少量数据、成本敏感的推理场景



● 竞价型实例

利用 EC2 的备用容量将成本降至最低。推荐用于容错和容忍中断的应用程序。了解竞价型实例

t4g.nano 的历史平均折扣为 67%

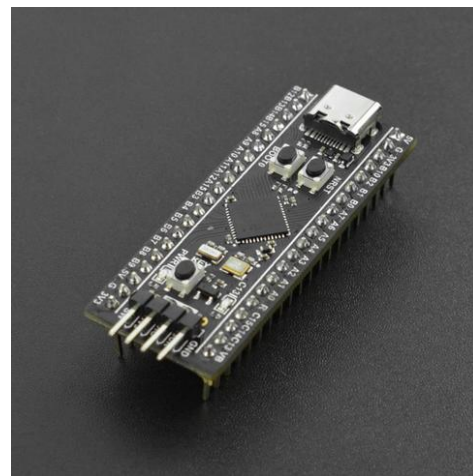
Assume percentage discount for my estimate

67

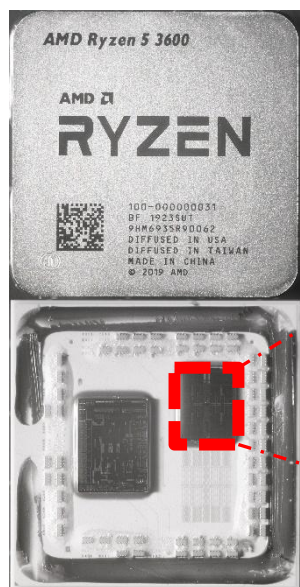
● 竞价型实例的实际定价各不相同

使用竞价型实例，您需要支付实例运行期间的有效竞价型价格

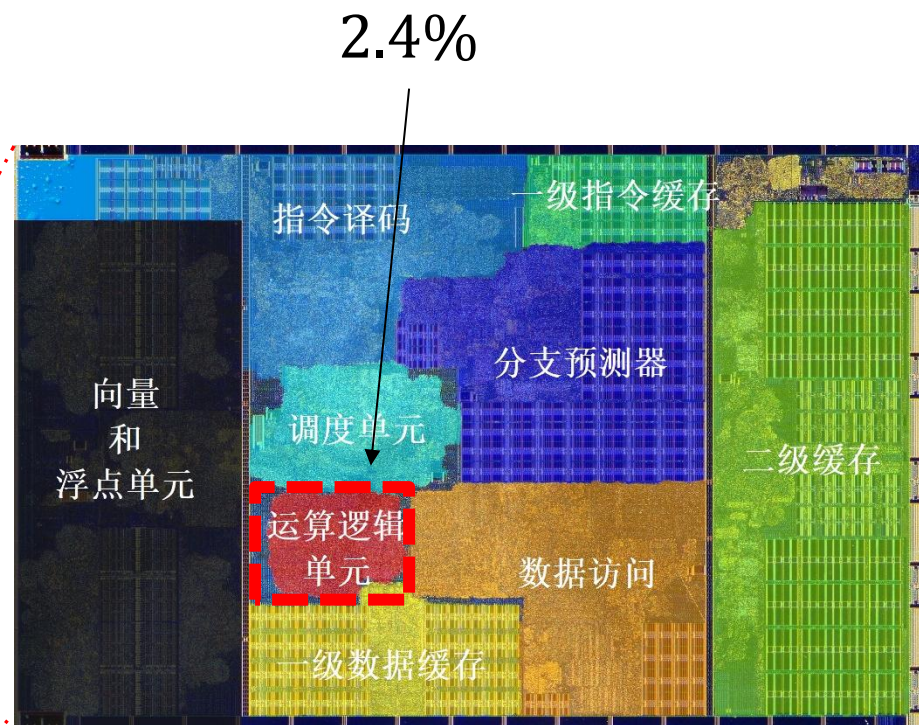
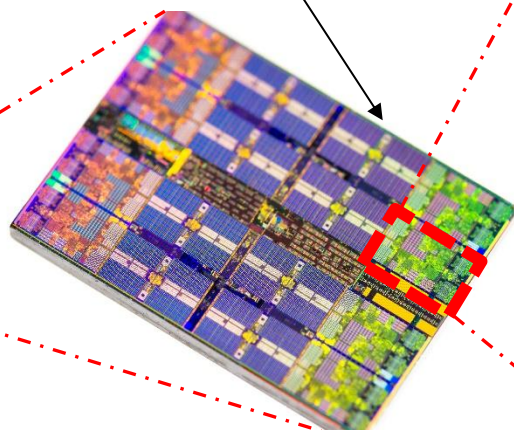
实例: \$0.0042/小时
每月: \$1.01/月



真实的处理器例子



$$5\% \times 8 = 39\%$$



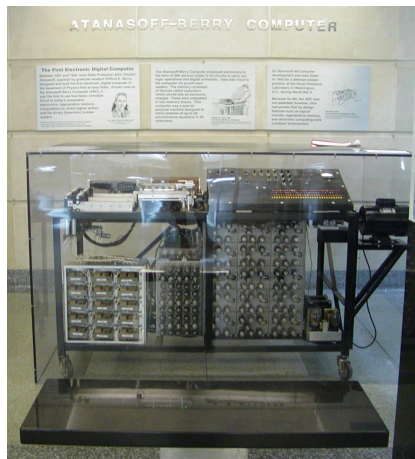
以AMD的锐龙3600处理器为例，用于运算的芯片面积不到
1%!

1930年代~1940年代

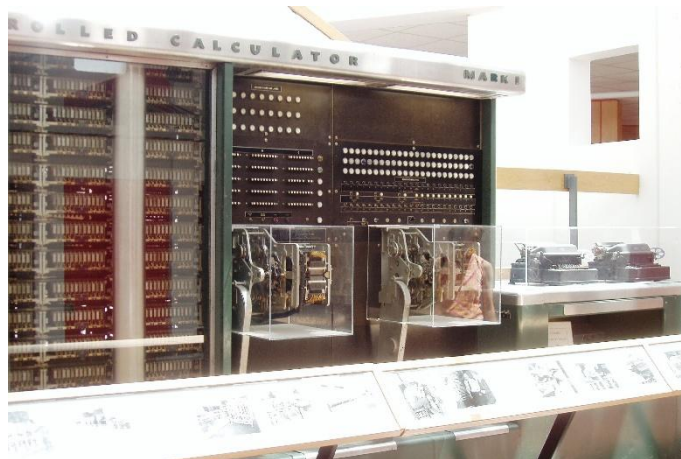
- ▶ 1936年 Alan Turing定义了图灵机和可计算性 (computability)

M., Turing, On Computable Numbers, with an Application to the Entscheidungsproblem, Proceedings of the London Mathematical Society, 1936.

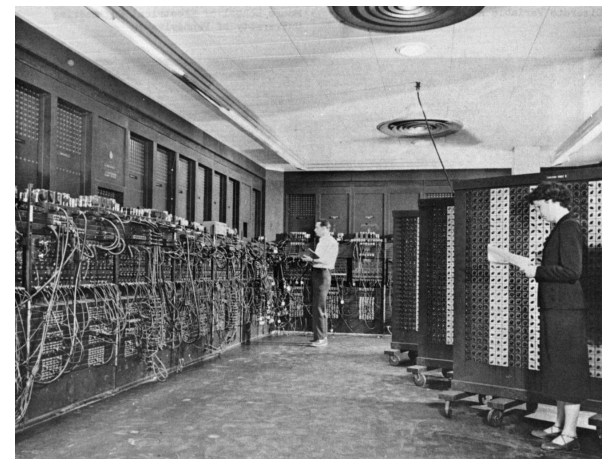
- ▶ 早期代表性计算机



1942 美国ABC
世界上第一台电子计算机



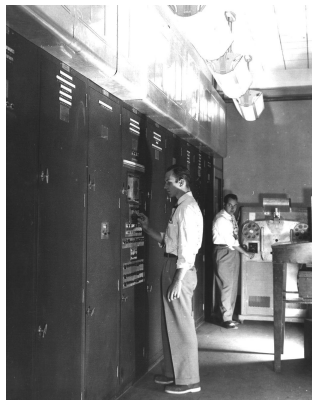
1944 美国Harvard Mark I



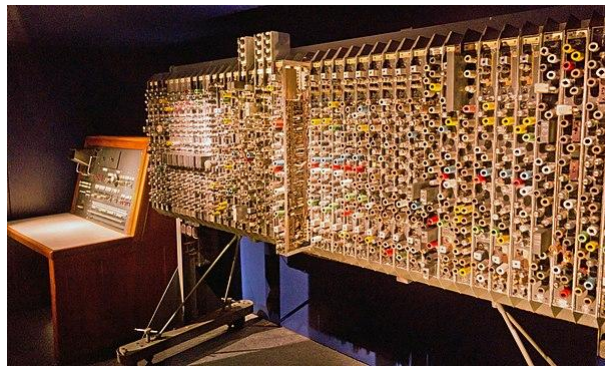
1945 美国ENIAC
世界上第二台电子计算机&第一台通用计算机

1940年代 ~ 1950年代

▶ 存储程序计算机



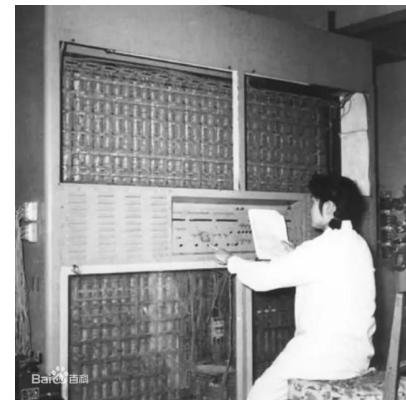
EDVAC, 冯诺依曼架构
1944-1951



英国ACE (Turing)
1946-1950



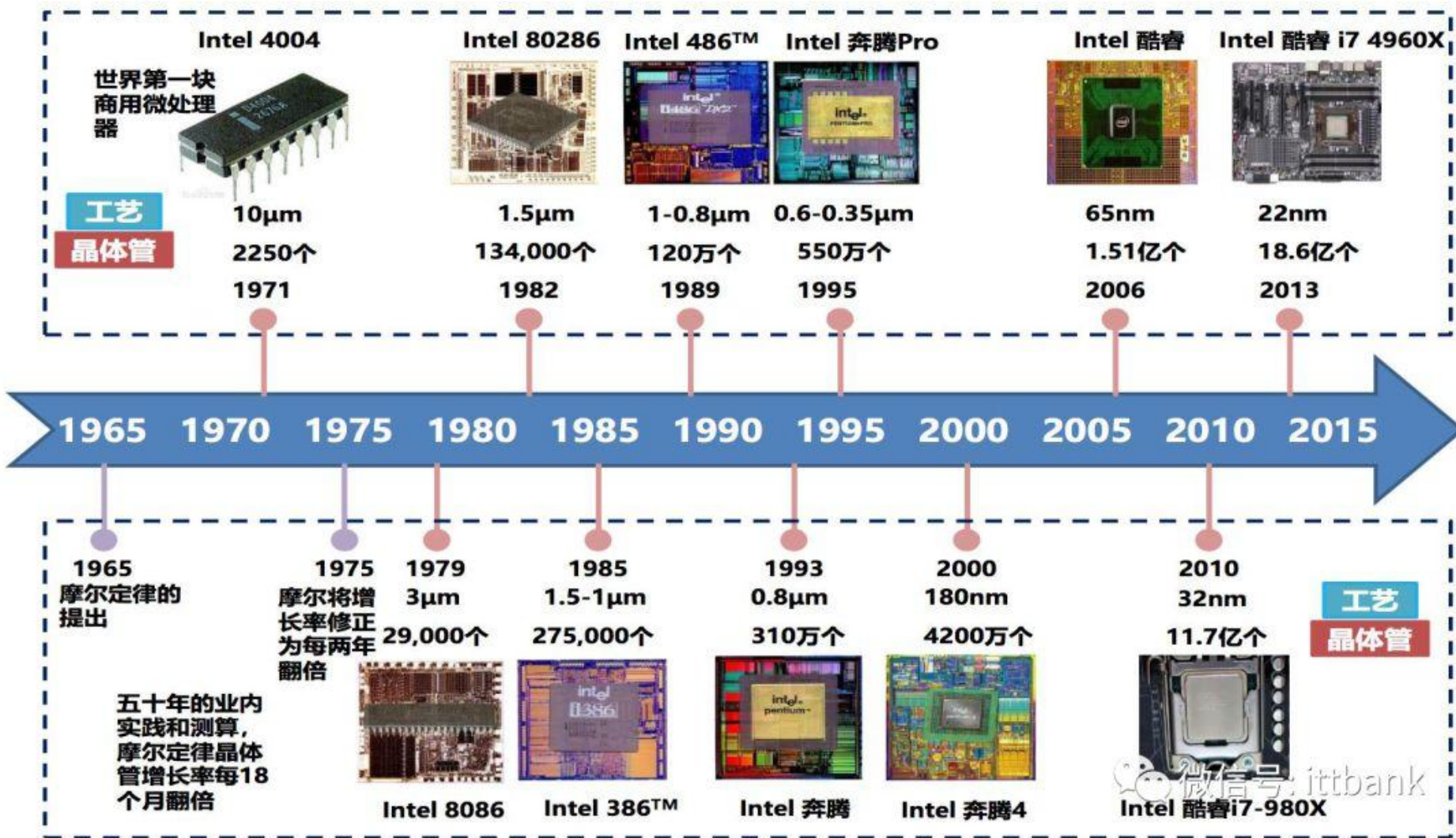
IBM 704, 第一台具有浮点运算硬件的量产计算机, 1954



103机, 1958年
中国第一台通用电子计算机

- ▶ 编程：机器语言 → 汇编语言
- ▶ 汇编器将符号代码和内存地址转换为机器代码

处理器的演进

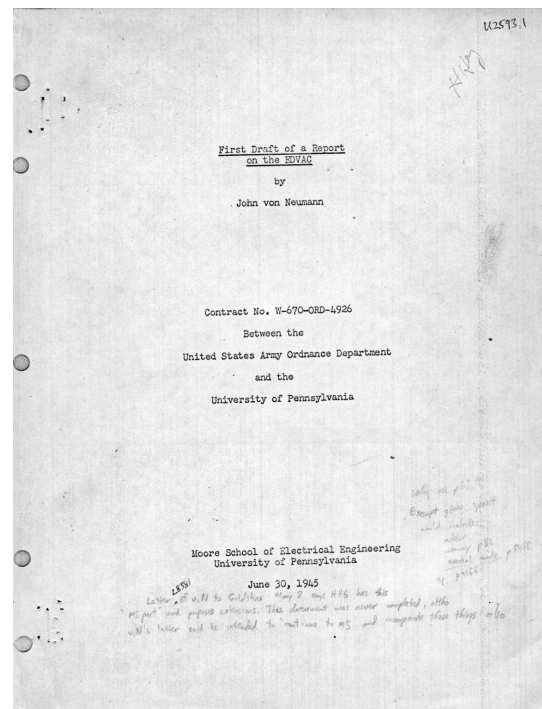
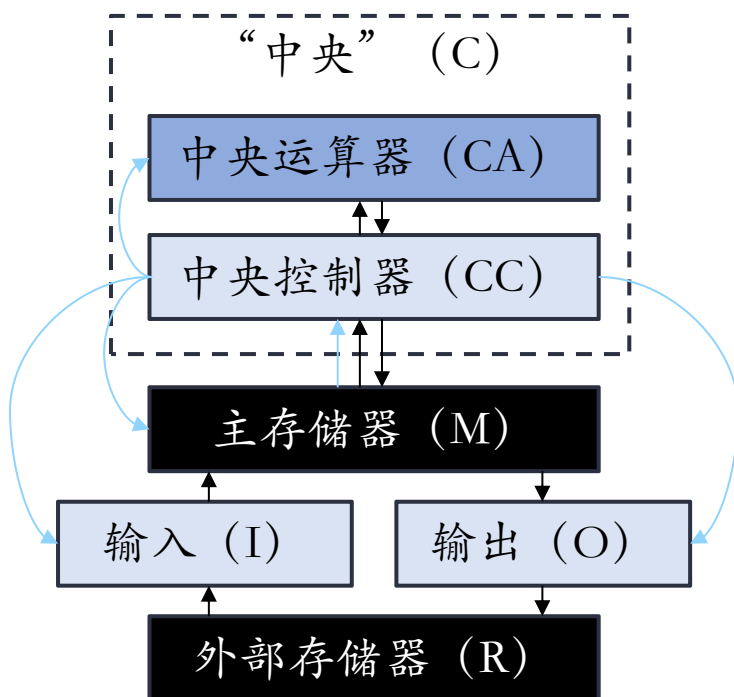


<https://www.eet-china.com/mp/a12970.html>

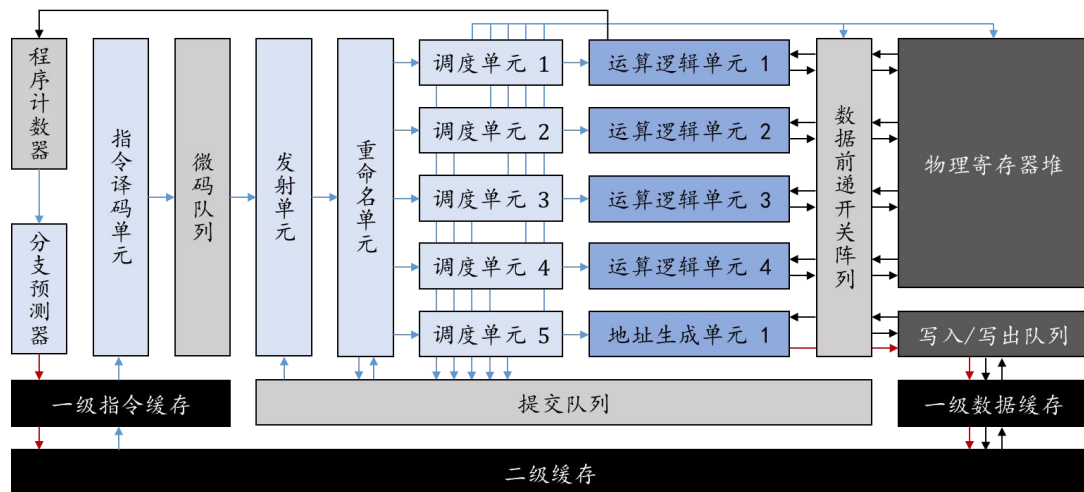
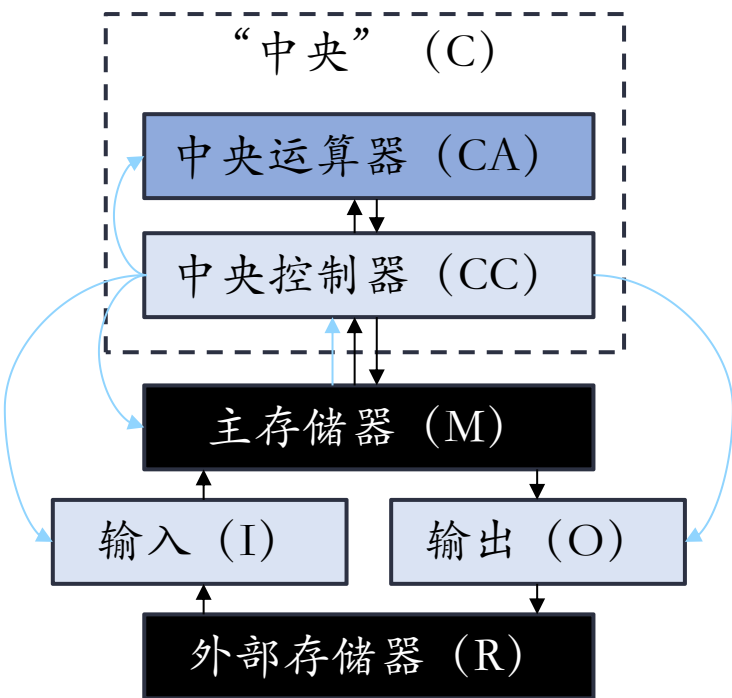
通用处理器结构

冯·诺依曼结构

- 包含控制器、运算器、存储器和输入/输出。
- “存储程序”：指令从主存储器中取出



通用处理器结构



冯·诺依曼结构

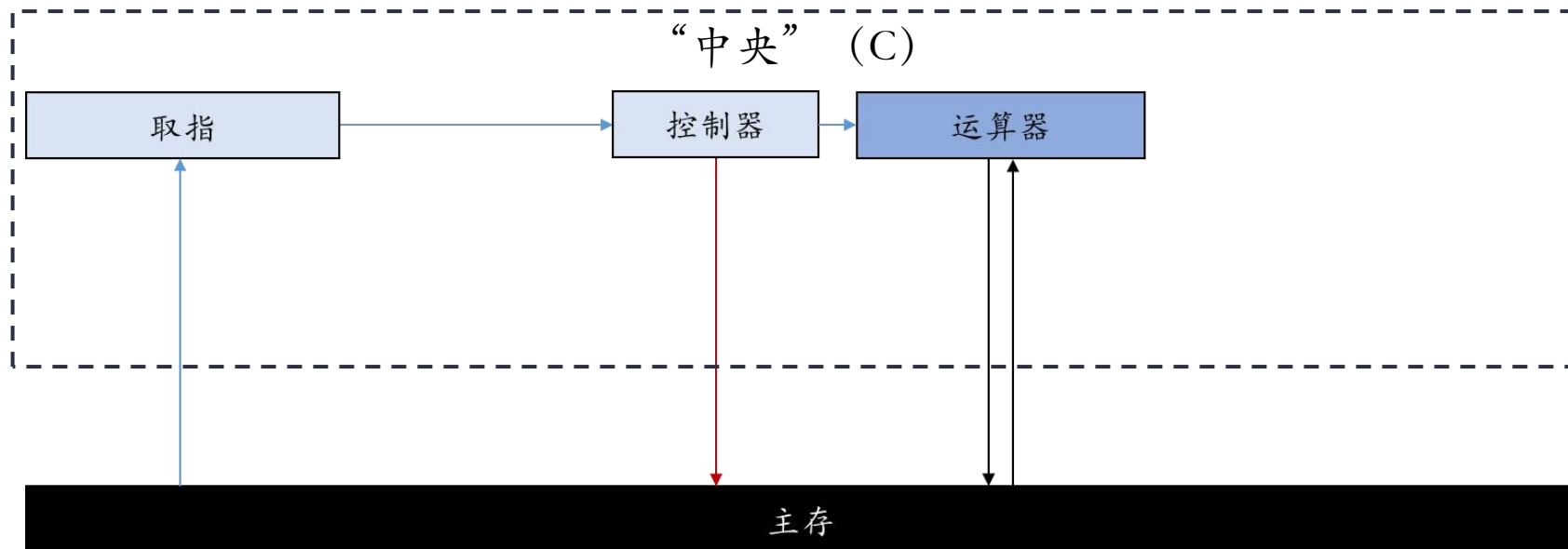
?

现代通用处理器CPU结构

通用处理器结构

冯·诺依曼结构

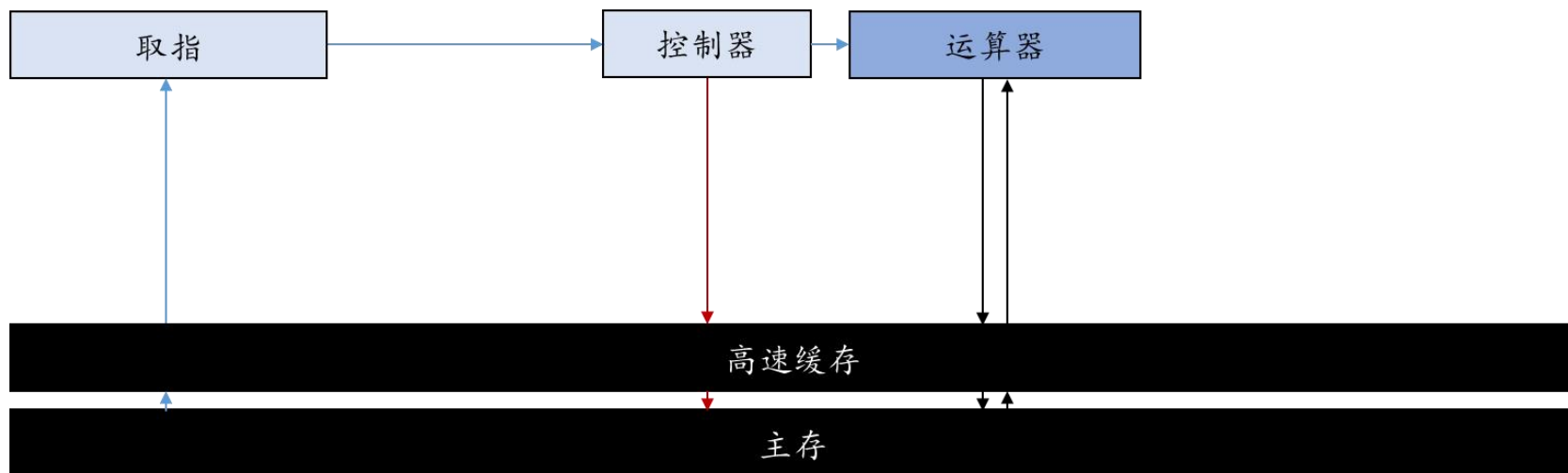
- ▶ 包含控制器、运算器、存储器和输入/输出。
- ▶ “存储程序”：指令从主存储器中取出



通用处理器结构

高速缓存

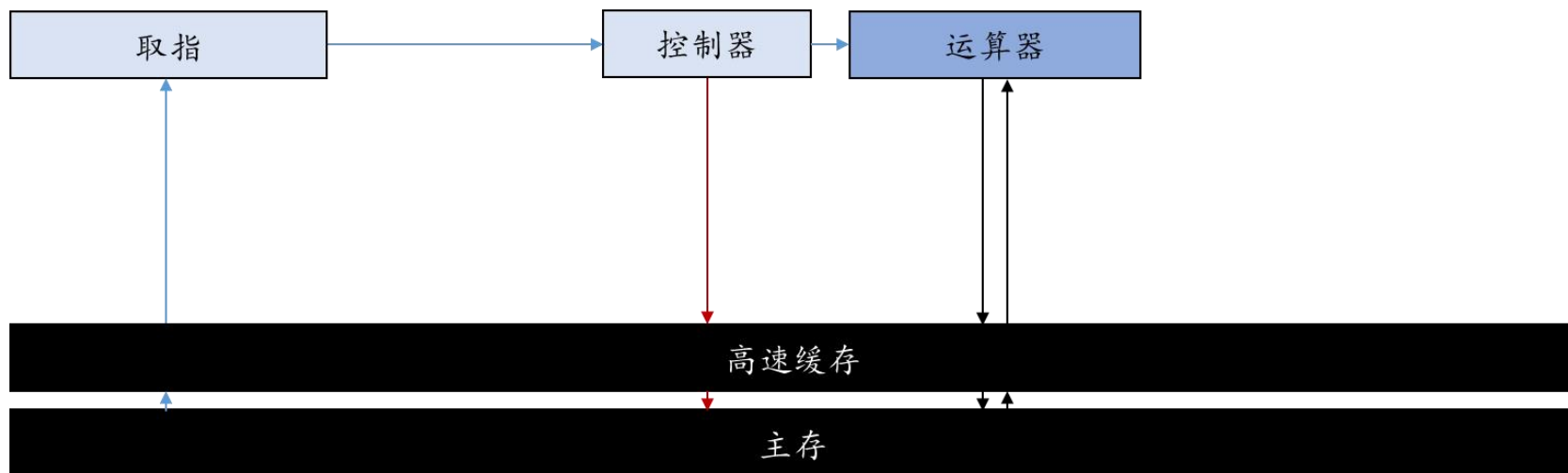
- ▶ 回顾：处理器的存储（ROM/DRAM/SRAM）
- ▶ 通用处理器的运算速度随着摩尔定律快速发展，但是主存的读写速度发展十分缓慢。



通用处理器结构

高速缓存

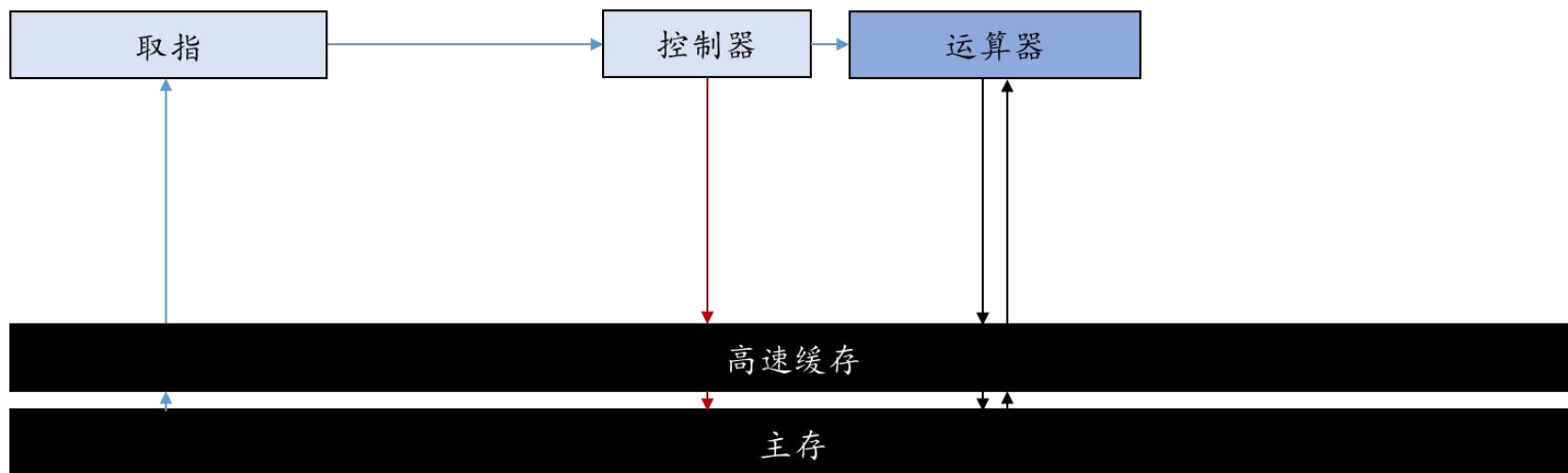
- ▶ 高速缓存用来弥补主存和运算器之间的“剪刀差”。
- ▶ 高速缓存位于处理器芯片上，常常采用昂贵的SRAM实现，和处理器的运算速度接近。



通用处理器结构

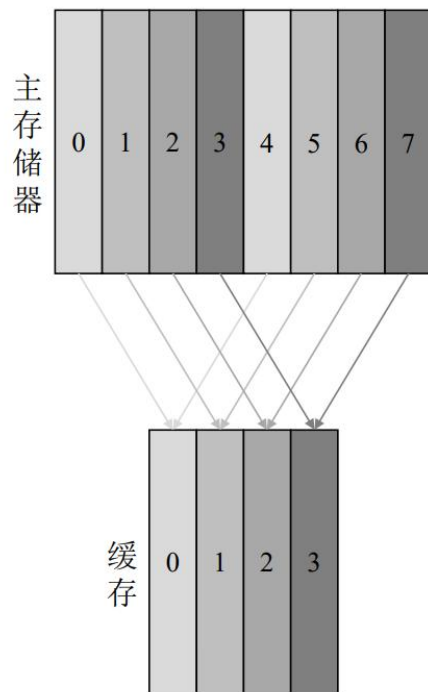
高速缓存

- ▶ 自动暂存最近读取的数据，以备不久之后再次使用
- ▶ 缓存硬件中设计了数据分配和替换机制，处理器访问主存时，会先经过缓存，若已经包含了该数据，则称为“命中”，由速度更快的缓存提供数据。

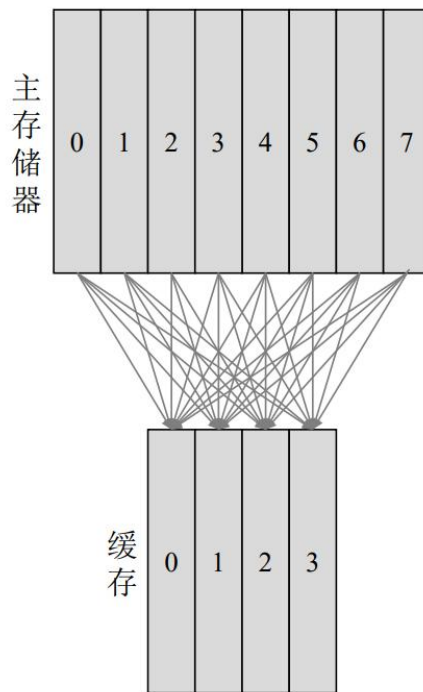


高速缓存

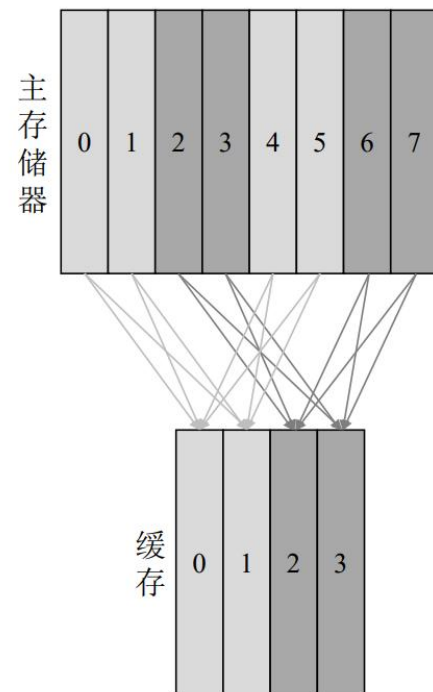
- 缓存有三种分配方式：直接相连、全相连、组相连



直接相连



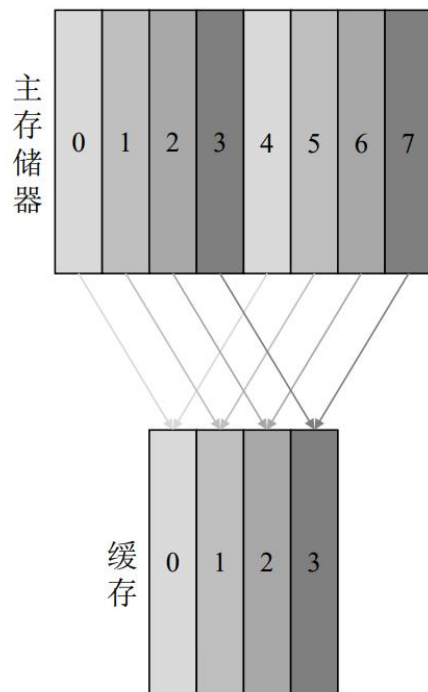
全相连



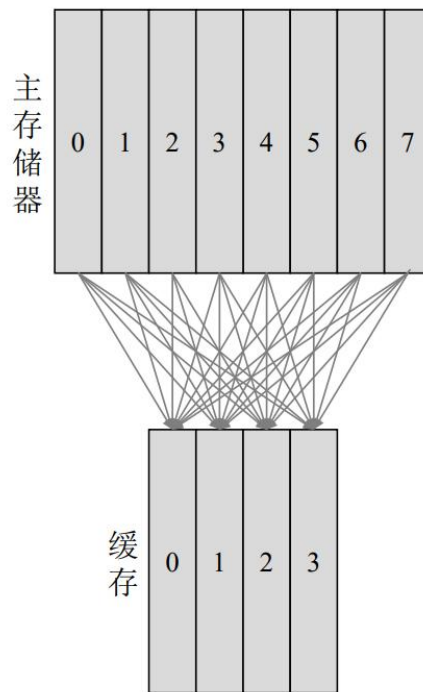
组相连

高速缓存

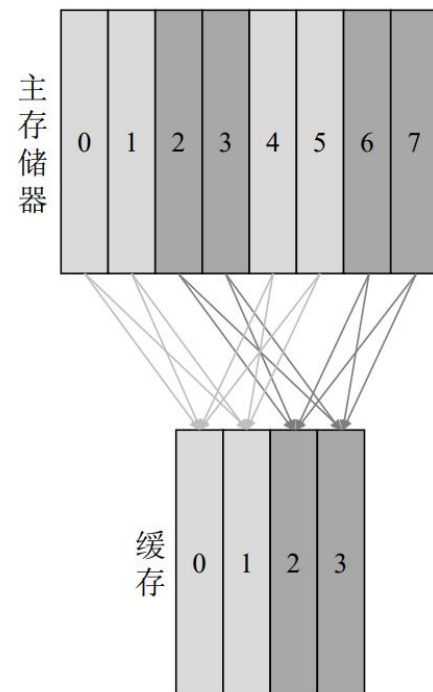
- ▶ 直接相连：将内存的数据按照地址分配到**固定位置**
- ▶ 全相连：允许将内存中的数据分配到缓存中的**任何位置**
- ▶ 组相连：两者的**折中**做法



直接相连



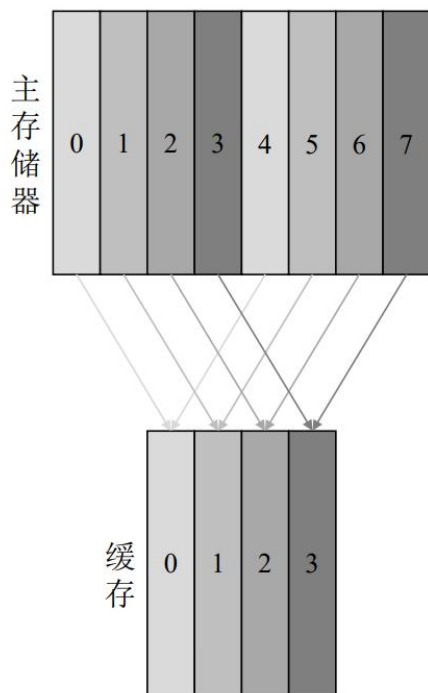
全相连



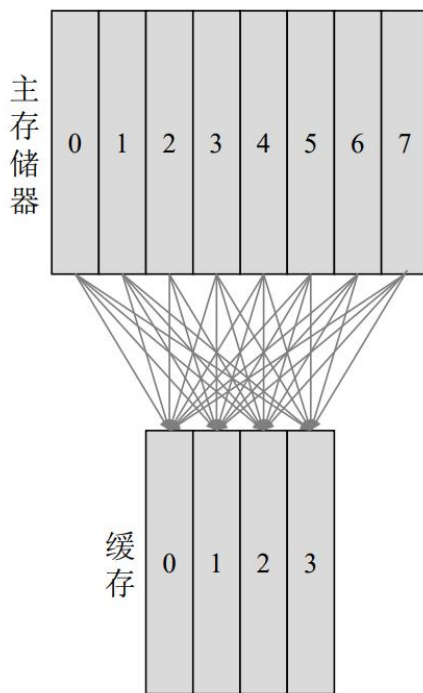
组相连

高速缓存

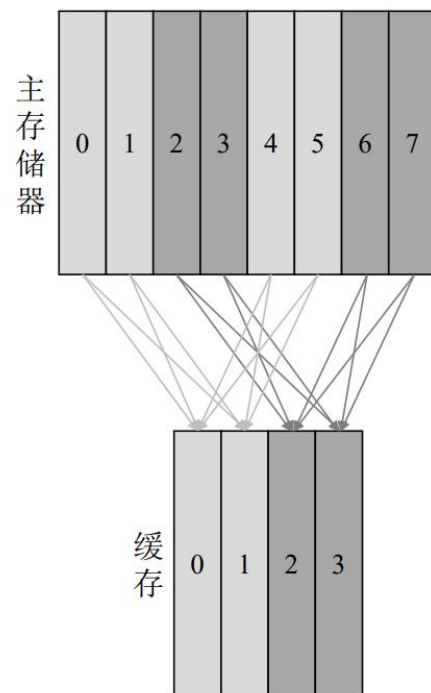
- 如果缓存数据已满，则会根据某种替换策略将已有数据替换出去，从而腾出空间来存储新的数据。
- 替换策略：随机替换、最长时间未用(LRU)、先进先出(FIFO)



直接相连



全相连

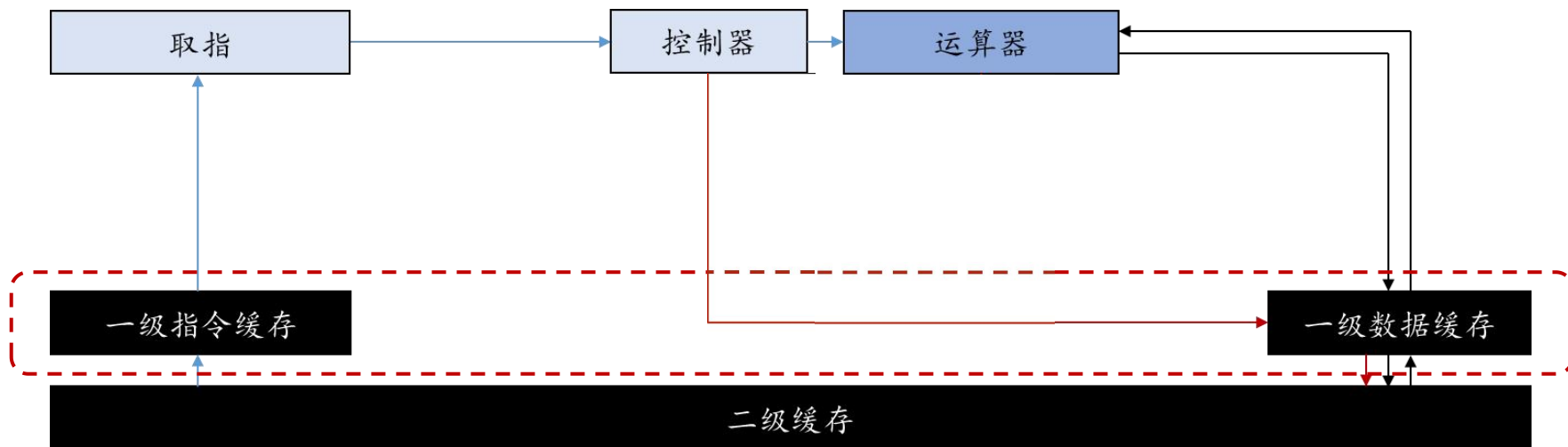


组相连

通用处理器结构

哈佛结构

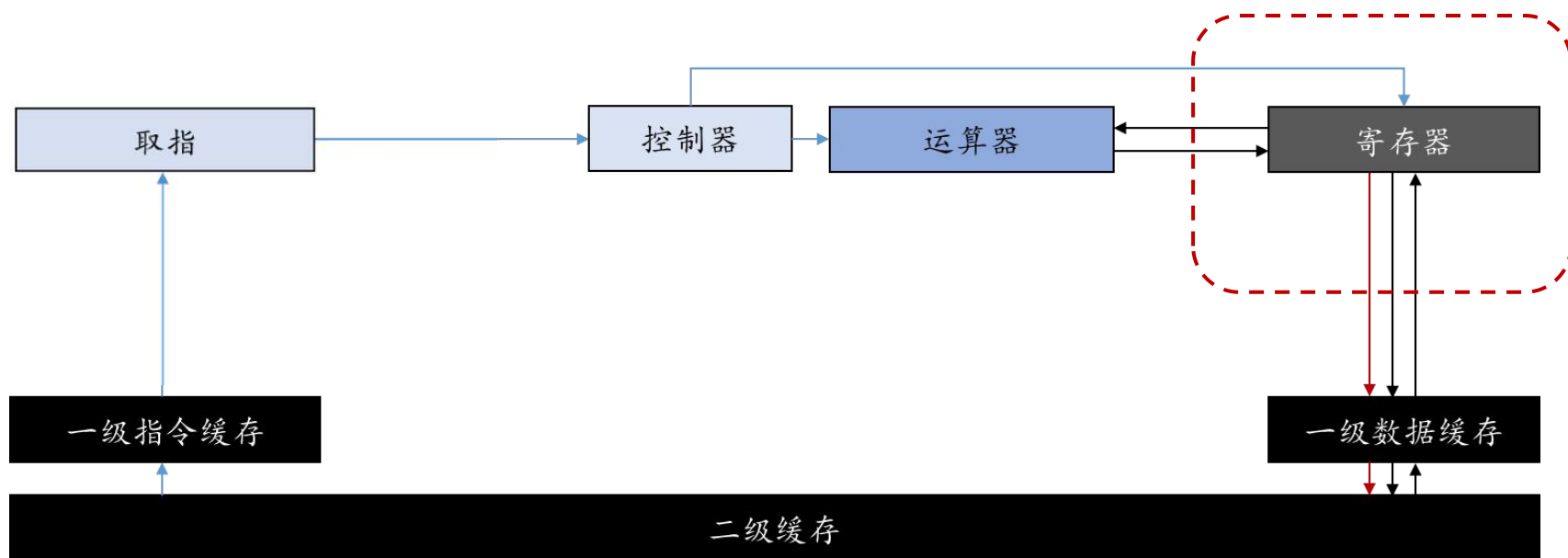
- ▶ 指令缓存和数据缓存分离
- ▶ 允许同时进行取指和访存，互不干扰



通用处理器结构

精简指令集结构（RISC）

- ▶ 关键原则：通过专门的load/store指令访存，所有计算指令都必须发生在寄存器上



指令集

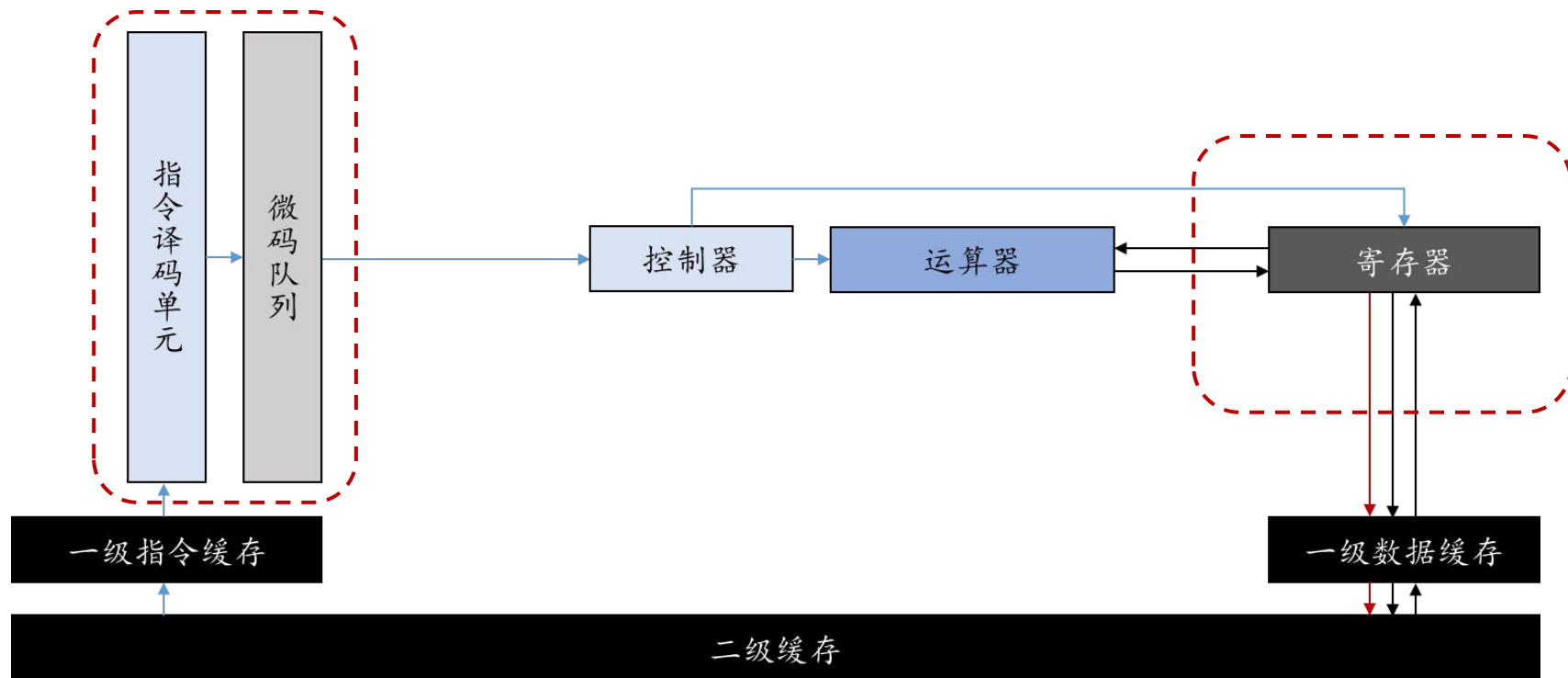
- ▶ 通用CPU指令集：分为RISC(Reduced Instruction Set Computer, 精简指令集计算机)、CISC(Complex Instruction Set Computer, 复杂指令集计算机)
- ▶ RISC: MIPS、ARM、RISC-V; 一条指令只完成简单任务

▶	CISC	RISC
出现时间	20世纪60年代	20世纪80年代
指令功能	复杂	简单
指令长度	多种指令长度	固定指令长度
指令数	多	较少
寻址模式	支持多种寻址模式	寄存器基址加偏移量

通用处理器结构

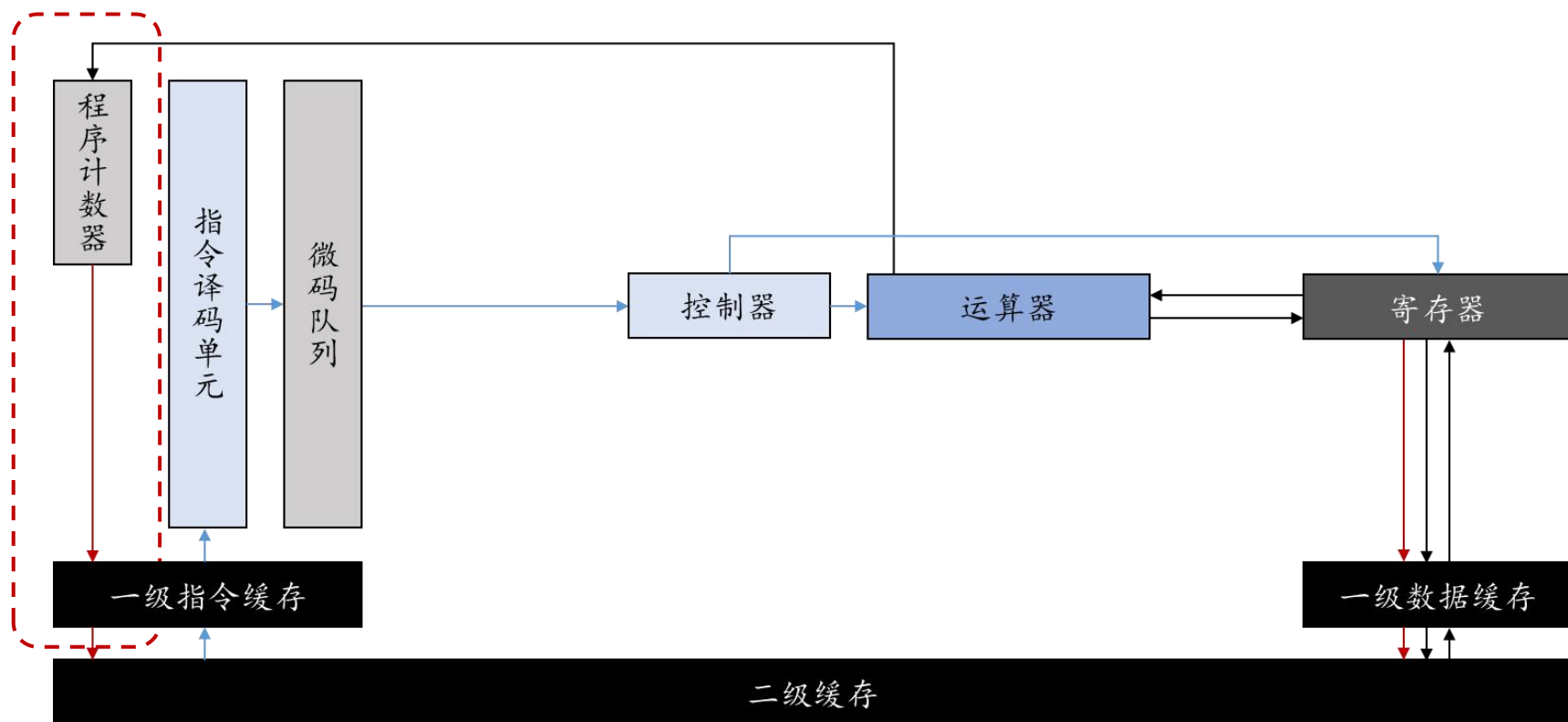
精简指令集结构（RISC）

- ▶ 实践中，以x86为例，会通过一个指令译码单元将复杂指令翻译为符合RISC原则的指令微码



通用处理器结构

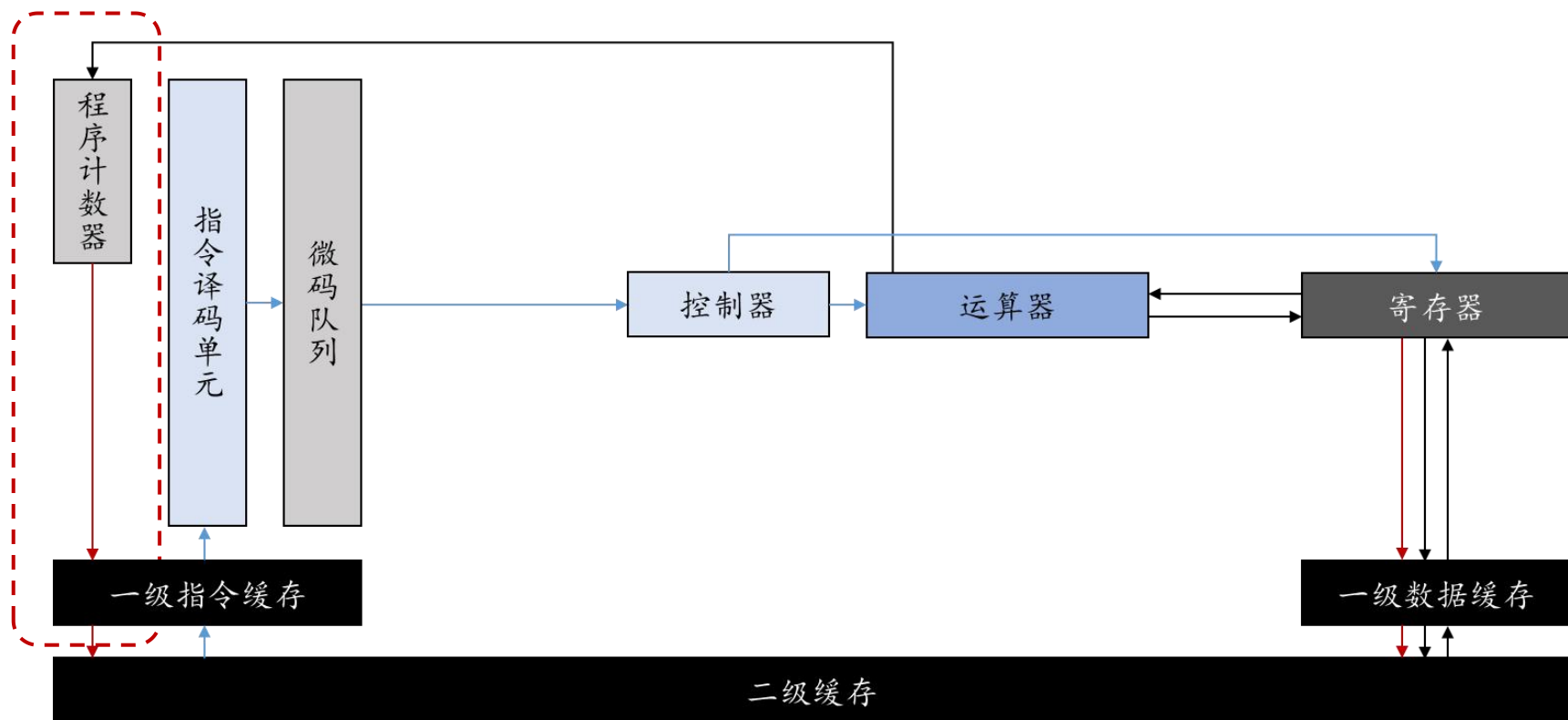
分支指令：原始的哈佛结构中，指令只能顺序执行，无法编写复杂的循环、分支操作。



通用处理器结构

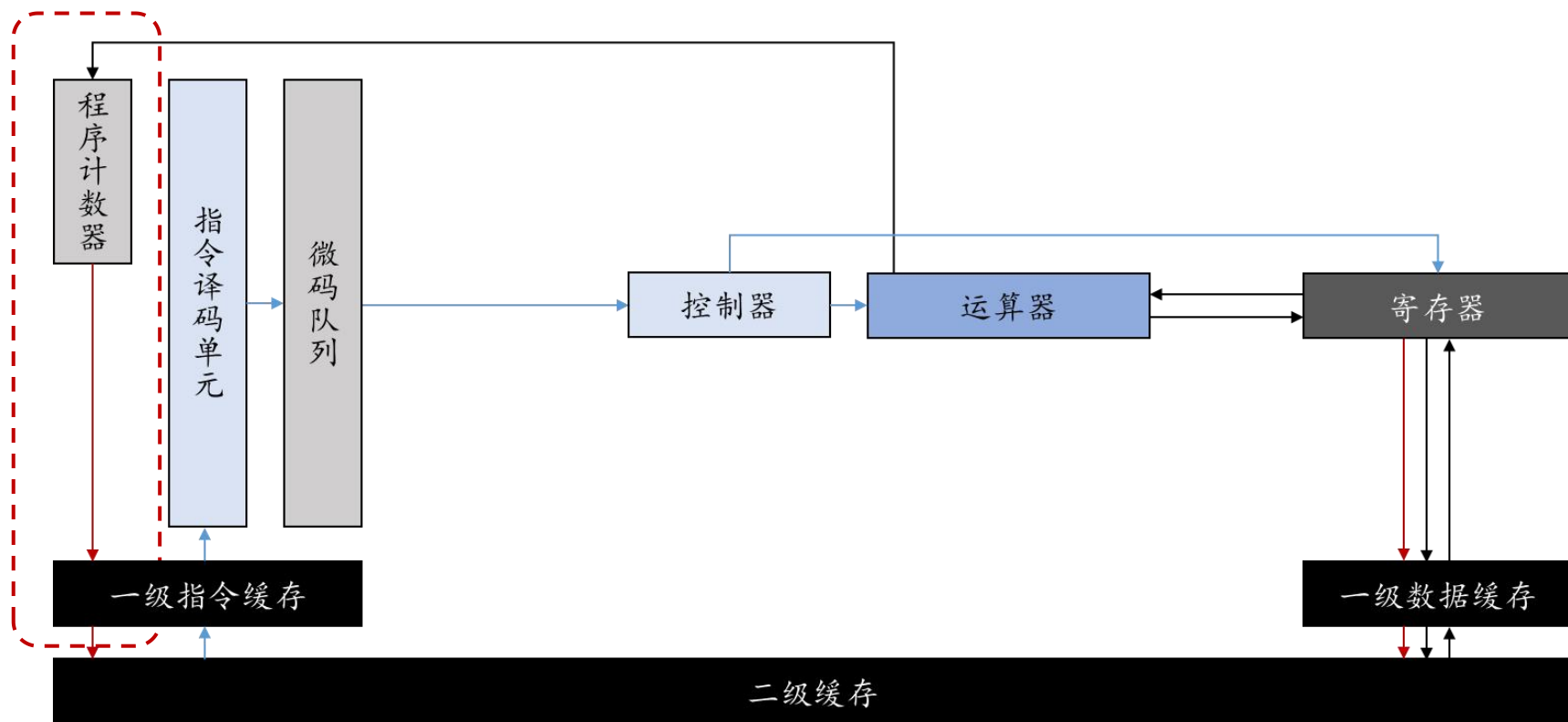
分支指令：由运算器计算出下一条指令的地址

- ▶ 分支指令计算完成前，暂停取指！



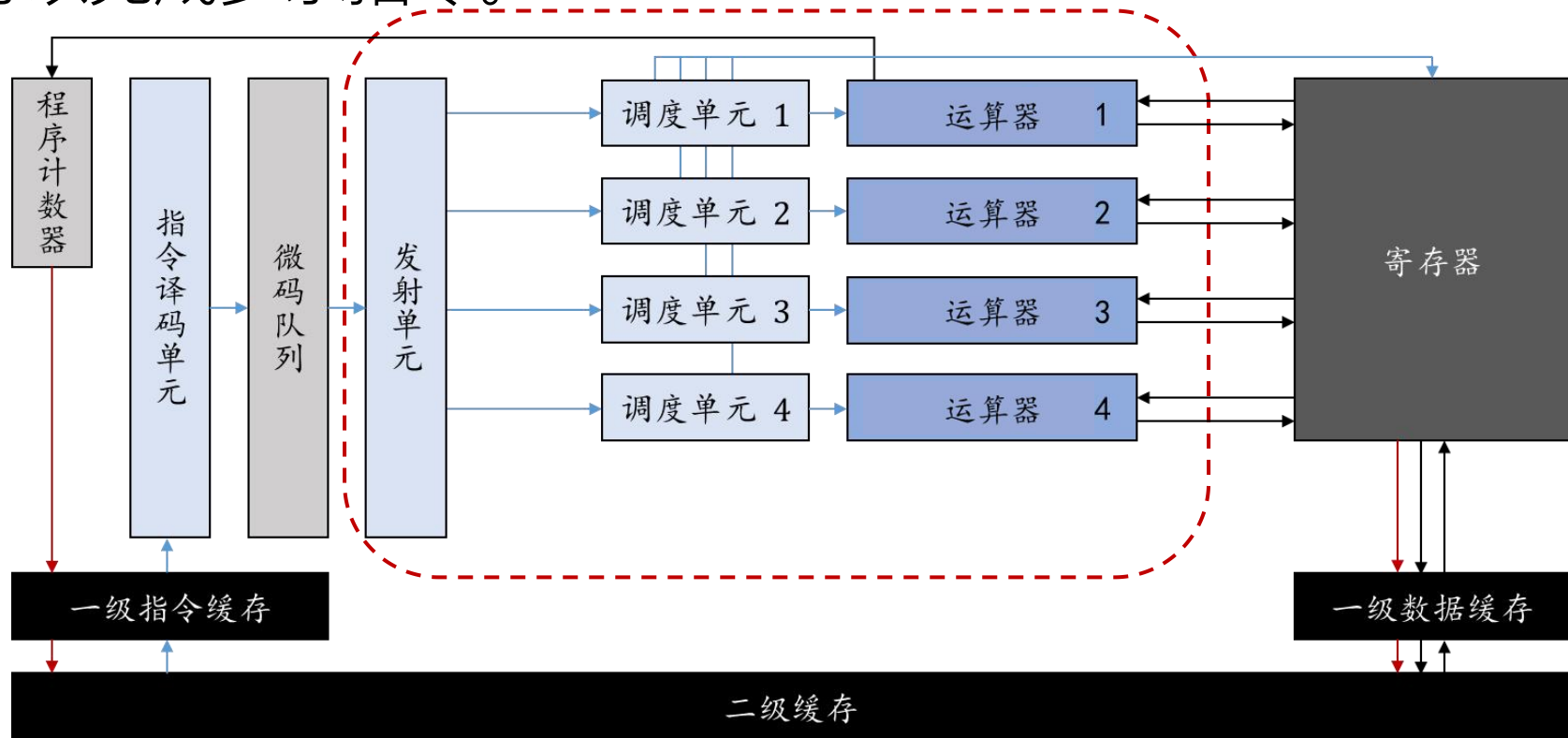
通用处理器结构-分支指令

分支指令：单独设计一个寄存器，称为程序计数器
(Program Counter, PC)，用于追踪下一条指令的地址。



通用处理器结构-多发射

多发射：指令微码送至运算器以备执行的过程称为发射。
为了提升指令的并行性，增加多个运算器，使得每个周期可以完成多条指令。

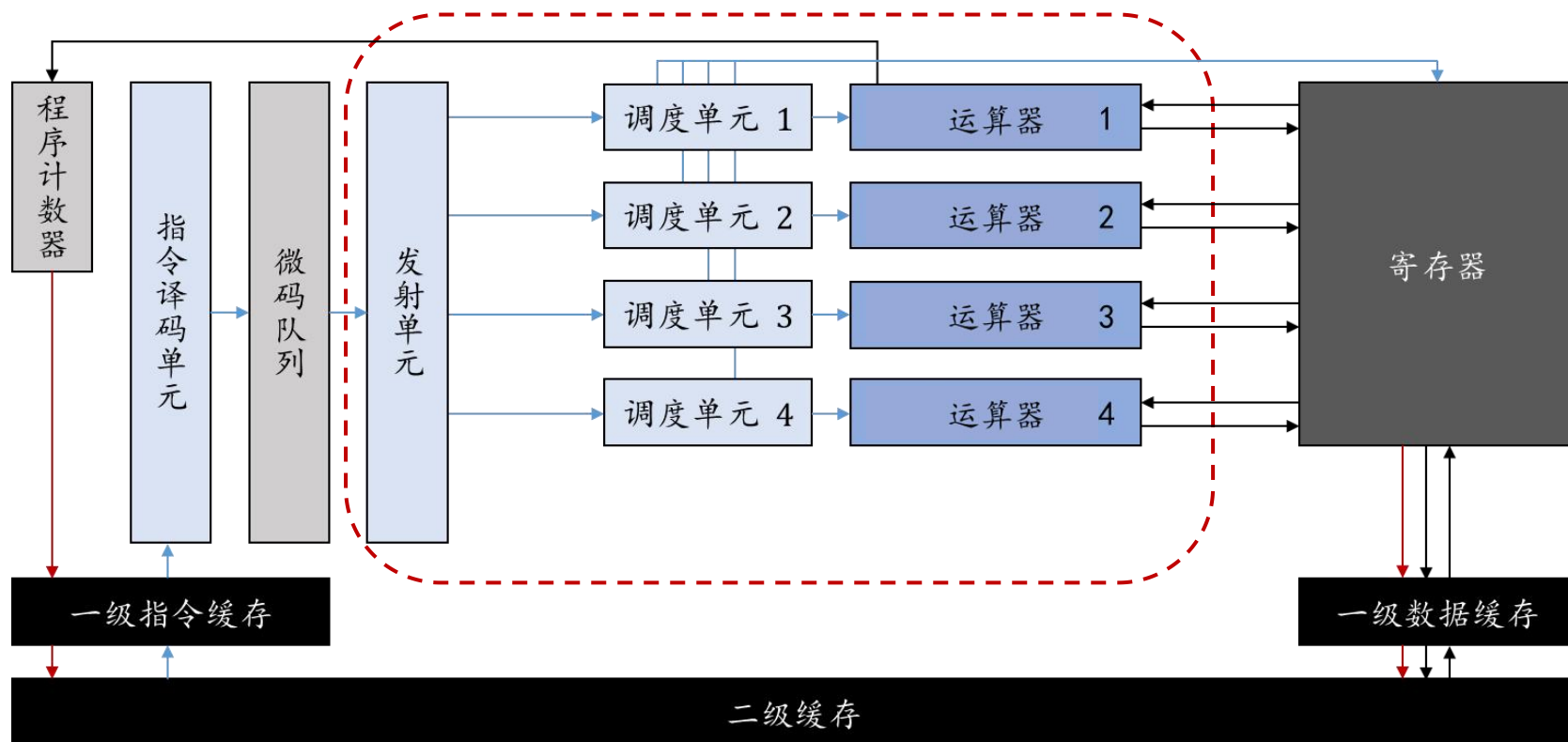


通用处理器结构-多发射

多发射：增加了运算器后，如果指令存在依赖，还是无法同时执行

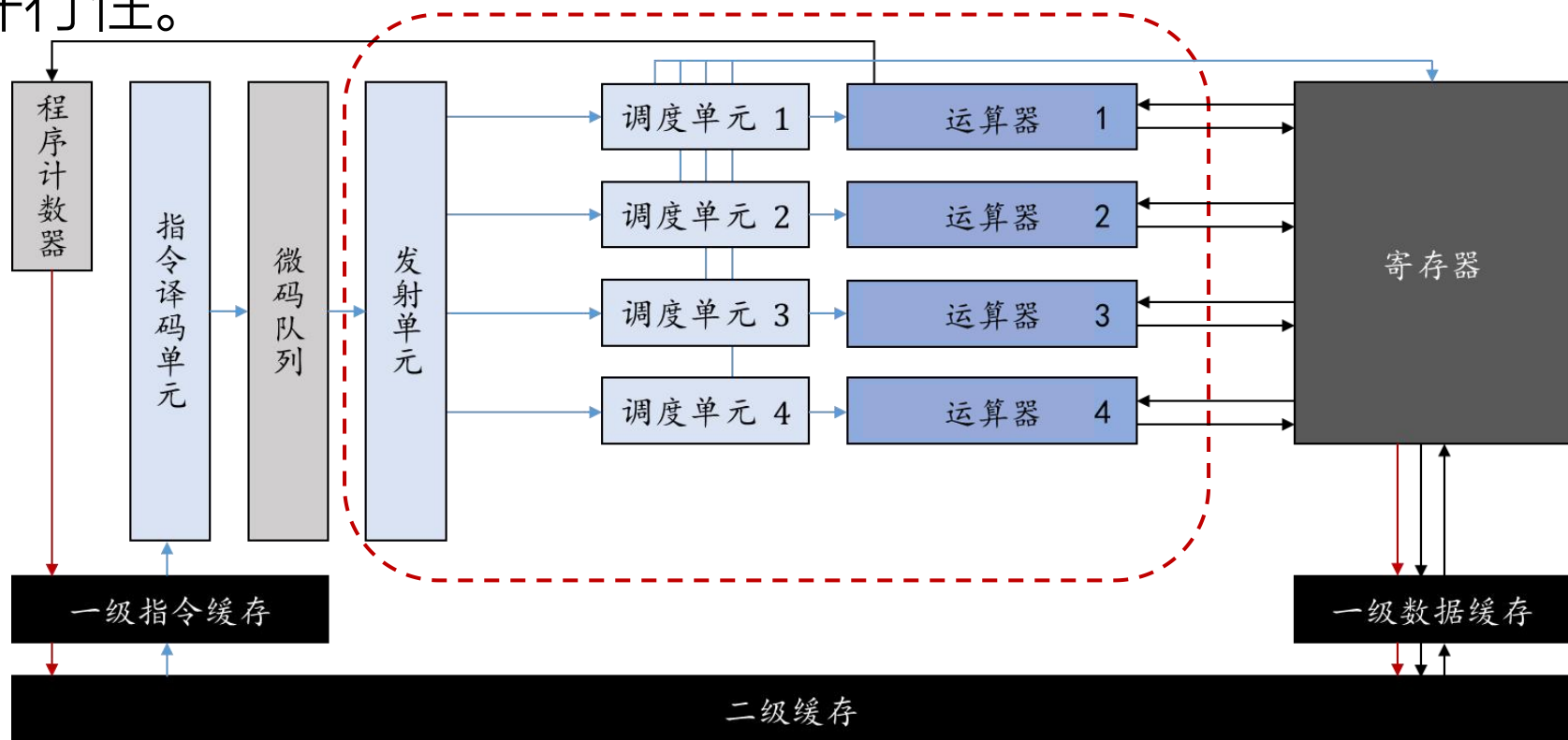
示例程序：

1. $r1 \leftarrow r2 \times r3$
2. $r4 \leftarrow r1 + r4$



通用处理器结构-多发射

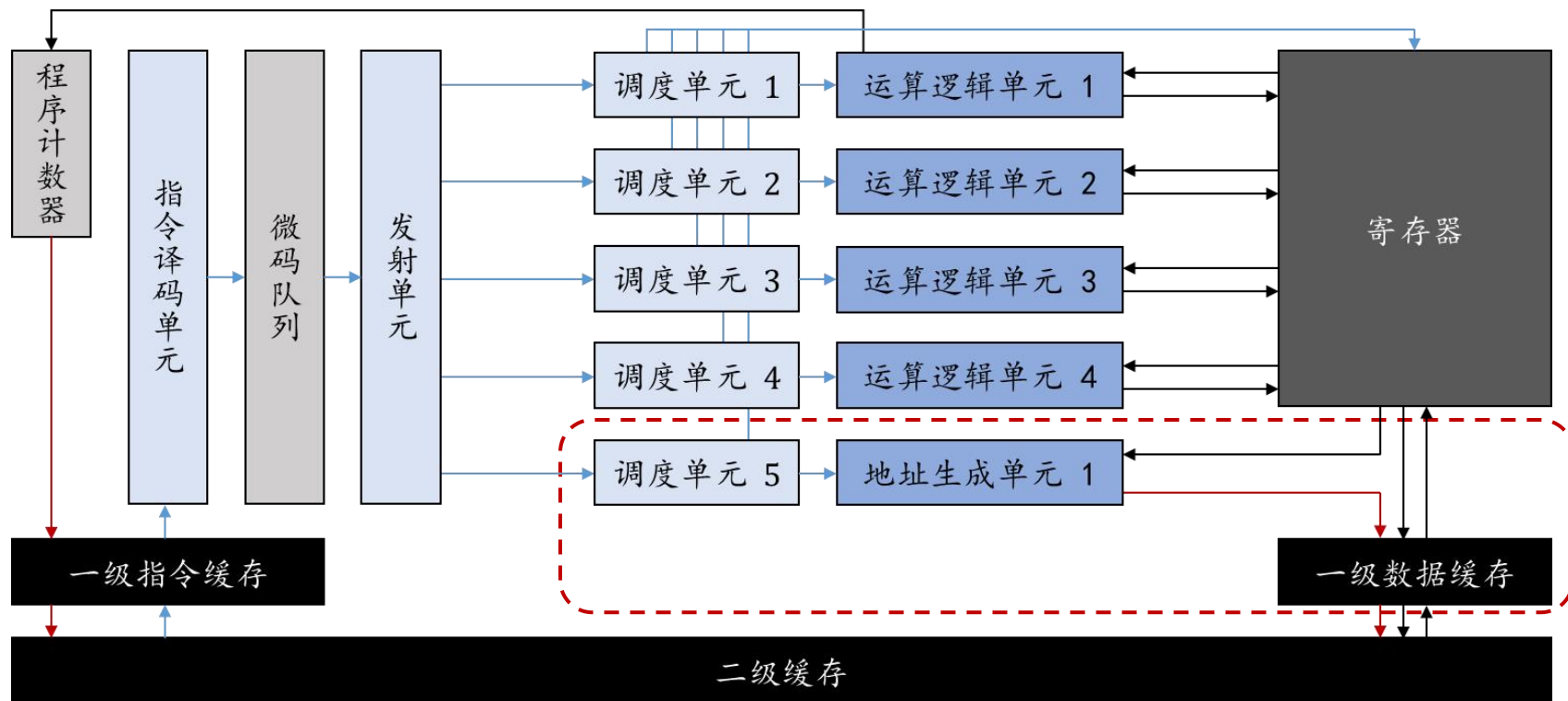
多发射：增加多个发射单元，多条互不相关的指令可以同时发射，这样可以同时利用多个运算器，提升指令的并行性。



通用处理器结构-地址生成单元

地址生成单元：专用于计算访存地址的运算器

- ▶ 寻址和运算是独立的，可以高效支持多种寻址模式（直接寻址、寄存器寻址、偏移寻址）



通用处理器结构-寄存器重命名

寄存器重命名：将寄存器编号与物理寄存器分开

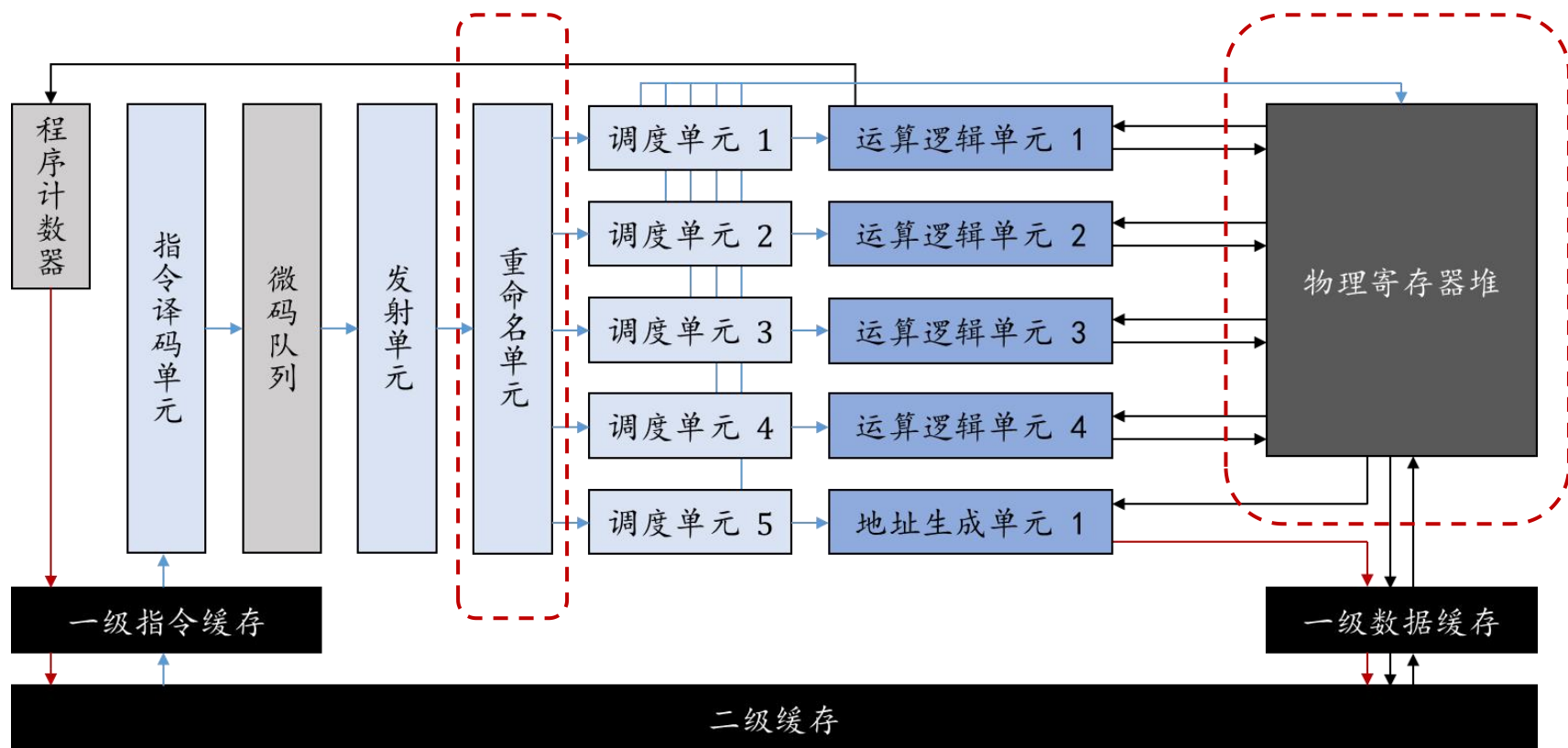
► 消除伪相关，提高指令同时执行的机会

示例程序：

1. $r1 \leftarrow r2 \times r3$
2. $r2 \leftarrow r3 + r4$

改为：

1. $r1 \leftarrow r2 \times r3$
2. $r5 \leftarrow r3 + r4$

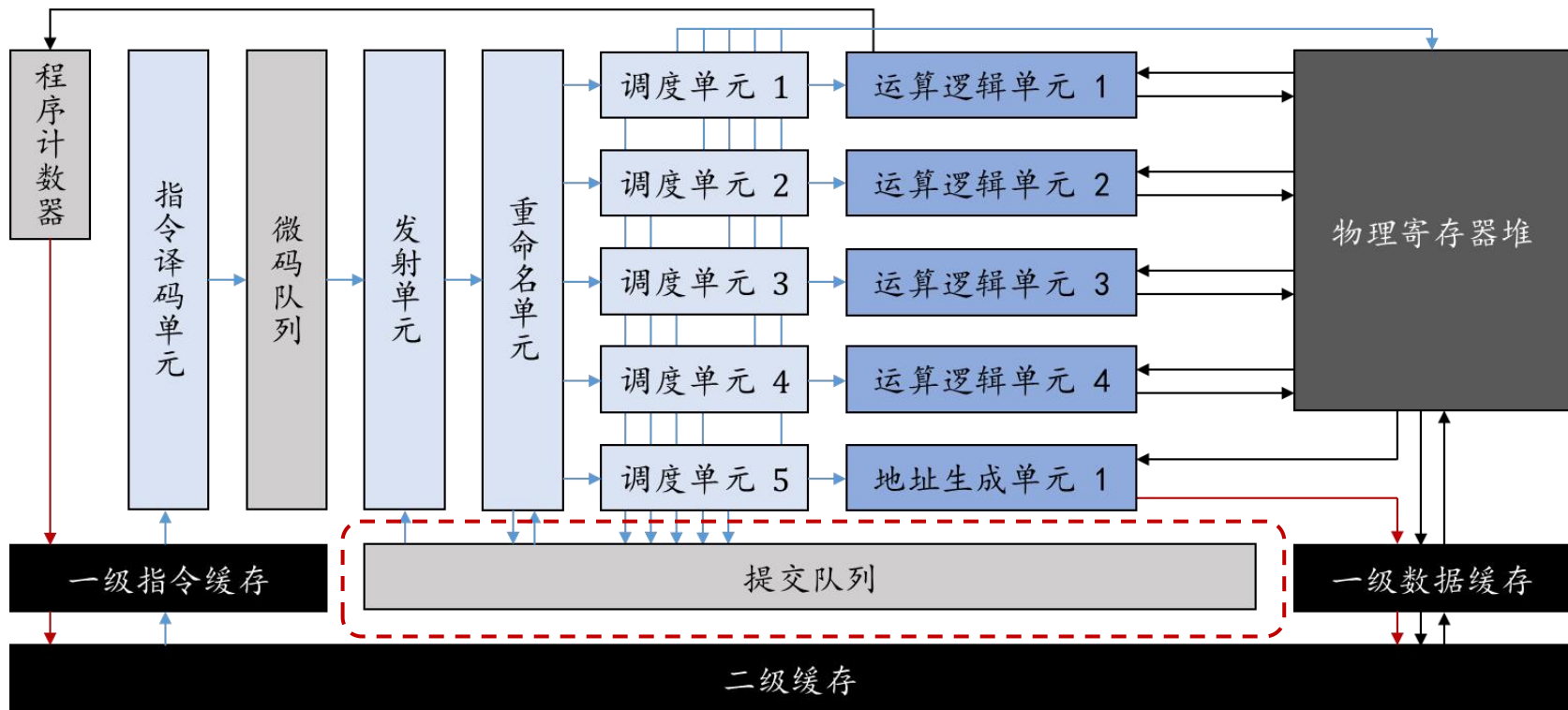


通用处理器结构-乱序执行

乱序执行：允许处理器在靠前的指令因为访存等待时，绕过该指令提前执行之后的无依赖指令。

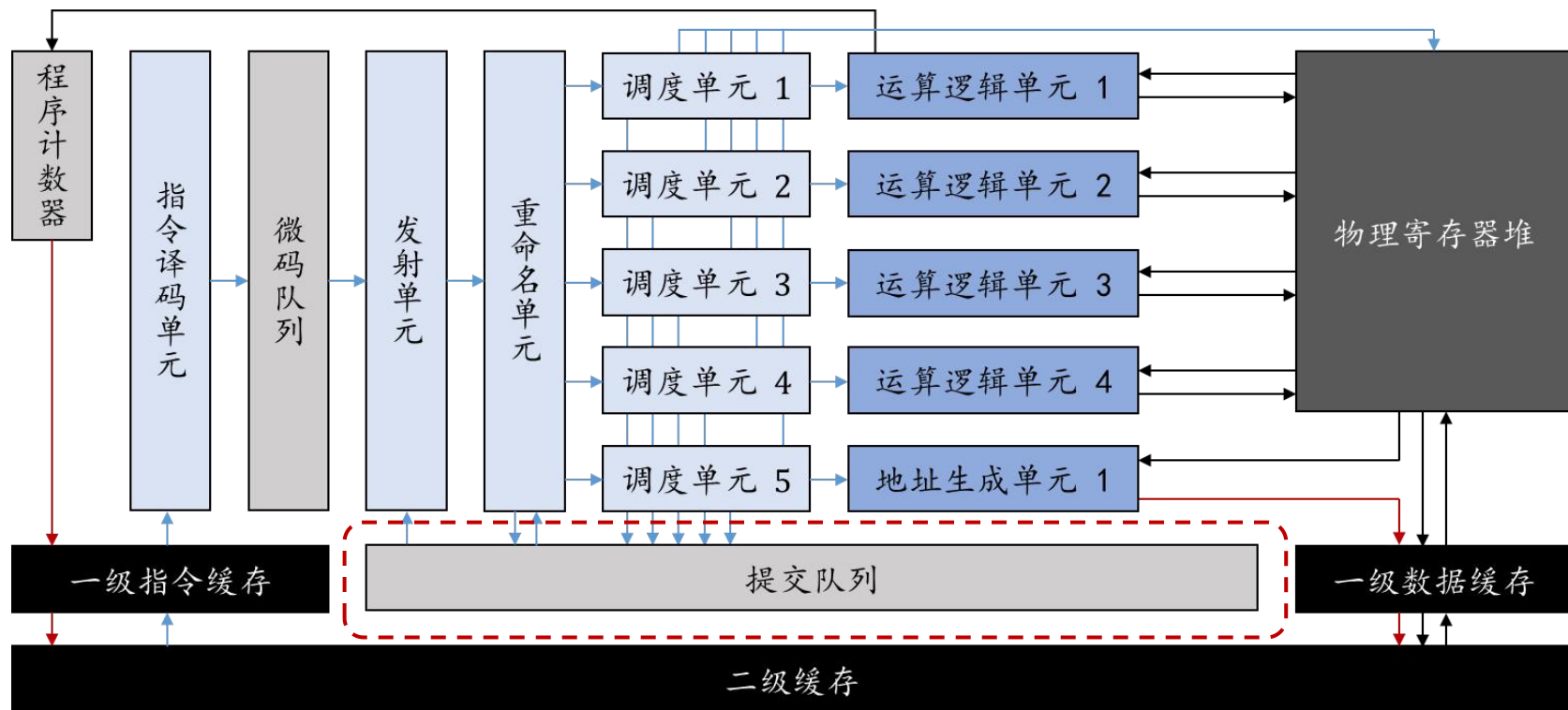
示例程序：

1. $r1 \leftarrow [r2]$
2. $r1 \leftarrow r1 + 1$
3. $r3 \leftarrow [r2 + 4]$



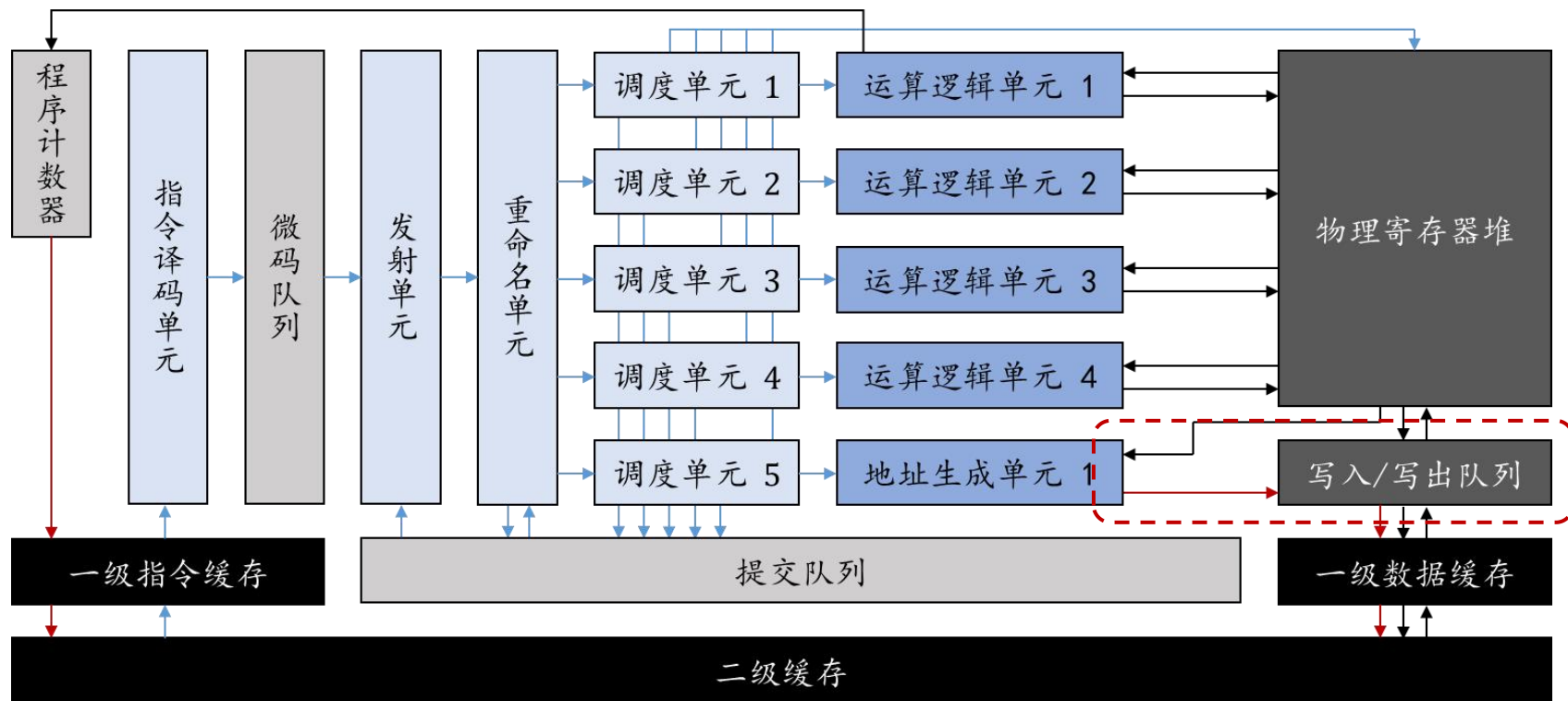
通用处理器结构-提交队列

- ▶ 如果指令出错或者发生跳转，导致靠后的指令本不该被执行，则需要撤销已执行的指令。
- ▶ 撤销机制通过提交队列实现，先存入队列，不要覆盖。



通用处理器结构-写入/写出队列

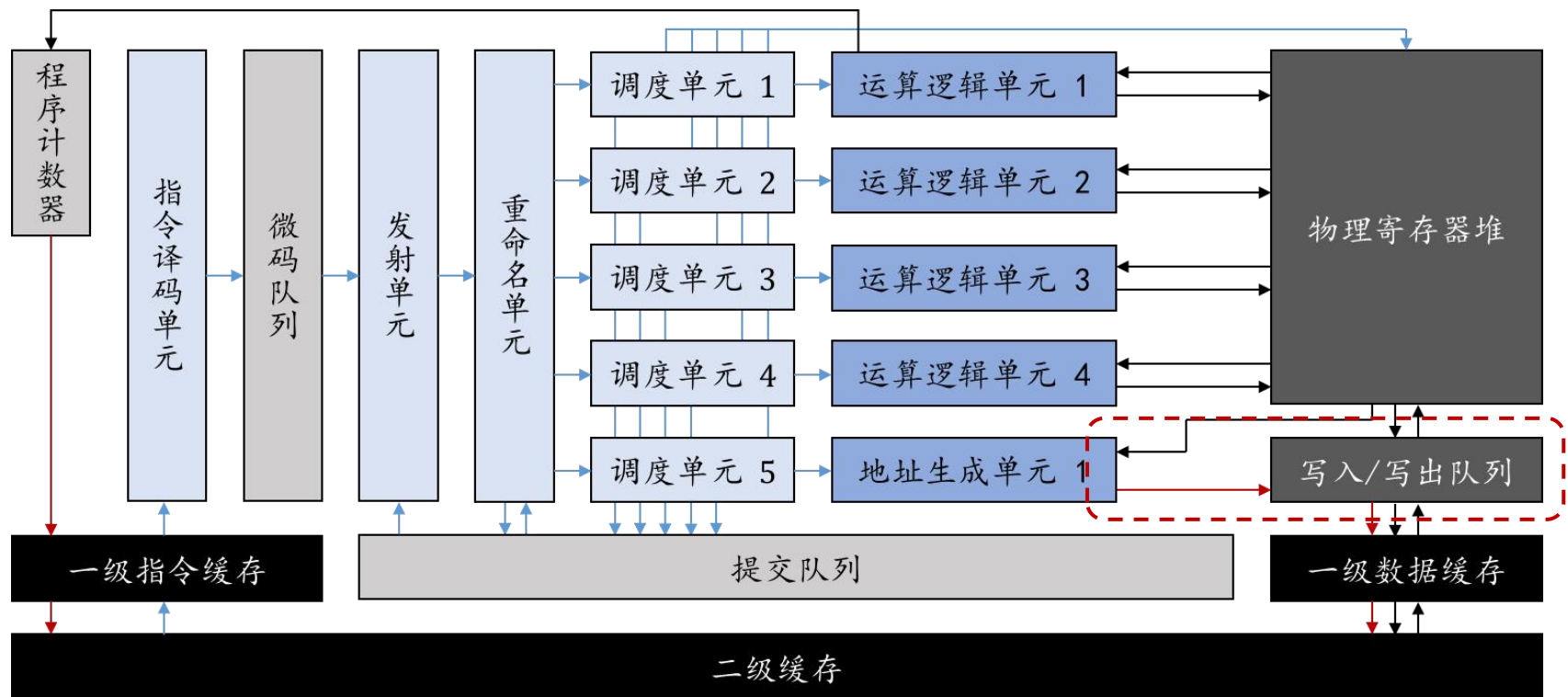
- 提交队列可以撤销处理器内部的指令，但是管理不了访存指令。



通用处理器结构-写入/写出队列

写入/写出队列：暂存已执行、未提交的访存指令

- ▶ 可以连续发起load/store指令，未提交前可以撤销



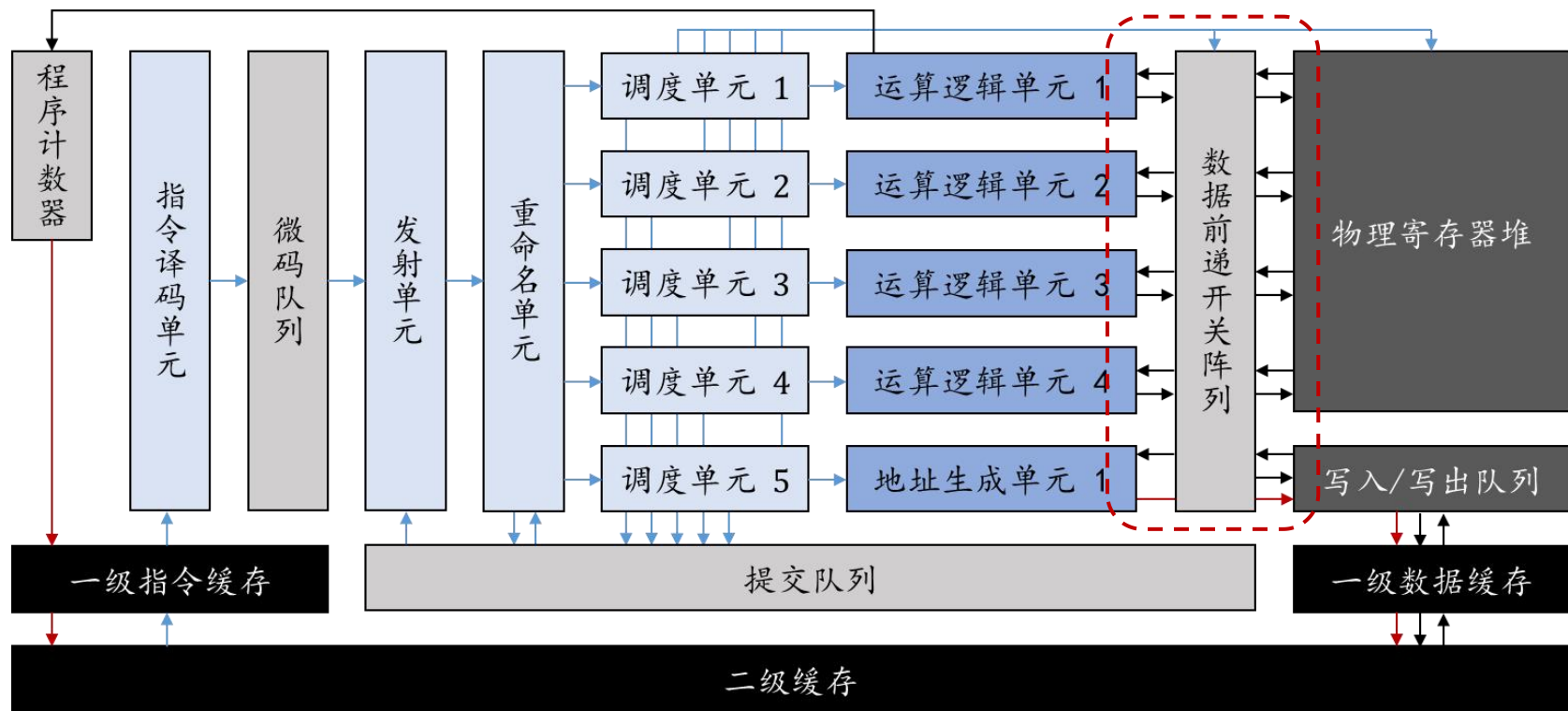
通用处理器结构-数据前递

示例程序:

1. $r1 \leftarrow r2 \times r3$
2. $r4 \leftarrow r1 + r4$

数据前递: 上一指令运算结果直接送入下一指令运算单元

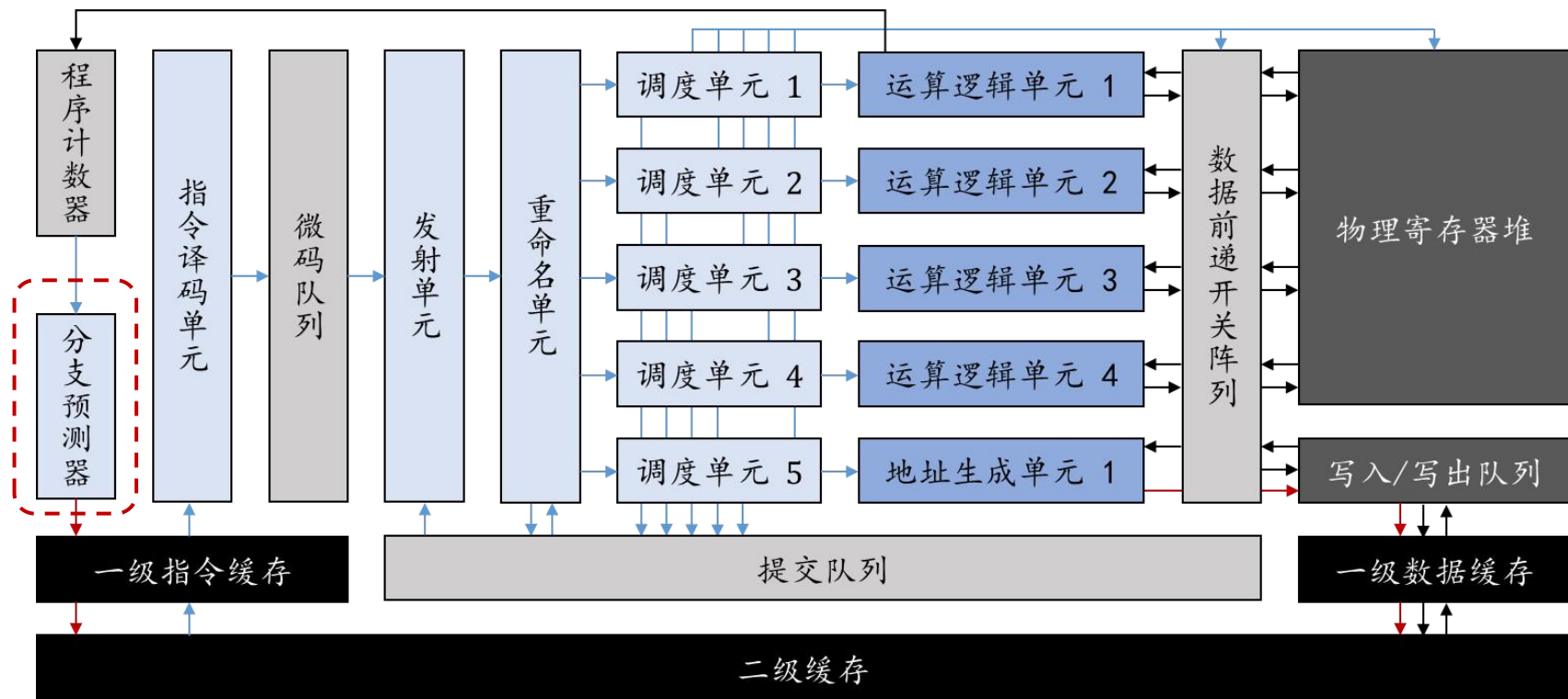
- ▶ 可以省去连续运算时反复写入、读出寄存器的动作 (抄近路)



通用处理器结构

分支预测：未确定跳转方向时，按猜测方向投机执行

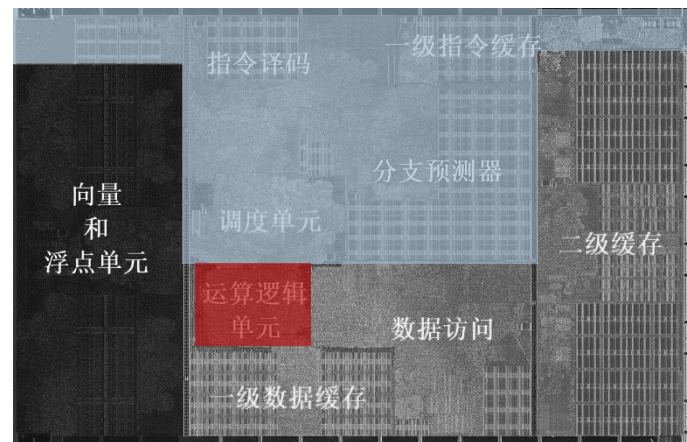
- ▶ 预测正确时：减少等待时间，提高了流水线效率
- ▶ 预测错误时：不予提交，撤销错误执行的指令（乱序执行的提交队列派上用场）



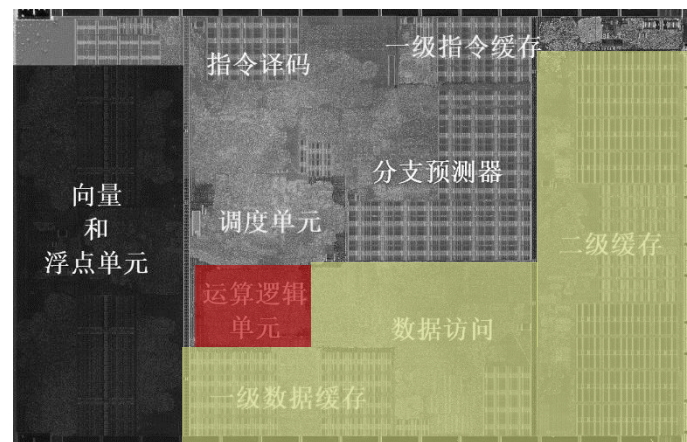
通用处理器结构

运算只占1%?

- ▶ 以标量作为基本运算粒度
 - ▶ 需要更多指令来执行
 - ▶ 任意指令间都潜在依赖，控制很复杂



- ▶ “内存墙”现象
 - ▶ 越来越大、越来越多层次的缓存



优化机会

卷积算子编译到指令集

```
.L2:    cmp     DWORD PTR [rsp-20], eax
      jle     .L1
      imul    rdx, rax, 27
      mov     QWORD PTR [rsp-40], r10
      xor     r9d, r9d
      mov     QWORD PTR [rsp-32], rdx
      xor     edx, edx
      mov     DWORD PTR [rsp-24], edx
.L12:   mov     edi, DWORD PTR [rsp-16]
      cmp     DWORD PTR [rsp-24], edi
      jge     .L16
      mov     rdx, QWORD PTR [rsp-40]
      xor     ebx, ebx
      xor     r11d, r11d
.L10:   mov     ecx, DWORD PTR [rsp-12]
      cmp     r11d, ecx
      jge     .L17
      movss   xmm0, DWORD PTR bias[0+rax*4]
      mov     r12, QWORD PTR [rsp-32]
      xor     r8d, r8d
      movss   DWORD PTR [rdx], xmm0
.L3:    mov     edi, DWORD PTR [rsp-8]
      cmp     r8d, edi
      jge     .L7
      lea     ecx, [r8+r9]
      lea     rbp, weight[0+r12*4]
      xor     edi, edi
      movsx   rcx, ecx
      imul    rcx, rcx, 224
.L8:    mov     r14d, DWORD PTR [rsp-4]
      cmp     edi, r14d
      jge     .L5
      lea     r13d, [rdi+rbx]
```

```
      mov     r14, rbp
      movsx   r13, r13d
      add     r13, rcx
      lea     r15, input[0+r13*4]
      xor     r13d, r13d
.L6:    cmp     r13d, DWORD PTR [rsp+56]
      jge     .L18
      movss   xmm0, DWORD PTR [r14]
      mulss   xmm0, DWORD PTR [r15]
      inc     r13d
      add     r14, 36
      addss   xmm0, DWORD PTR [rdx]
      add     r15, 200704
      movss   DWORD PTR [rdx], xmm0
      jmp     .L6
.L18:   inc     edi
      add     rbp, 4
      jmp     .L8
.L5:    inc     r8d
      add     r12, 3
      jmp     .L3
.L7:    inc     r11d
      add     rdx, 4
      add     ebx, esi
      jmp     .L10
.L17:   inc     DWORD PTR [rsp-24]
      add     r9d, esi
      add     QWORD PTR [rsp-40], 888
      jmp     .L12
.L16:   inc     rax
      add     r10, 197136
      jmp     .L2
.L1:    
```

控制

寻址

计算

► 控制和寻址开销超过了实际的计算!

控制、寻址

- ▶ 降低控制开销：循环展开 (Loop unrolling)
- ▶ 也可以通过部分展开保留循环结构的控制能力

```
1. r3 ← 0
2. r1 ← r1 × r2
3. r3 ← r3 + 1
4. 跳转至 2., 如果 r3 < 8
```

共执行25条指令

```
1. r1 ← r1 × r2
2. r1 ← r1 × r2
3. r1 ← r1 × r2
4. r1 ← r1 × r2
5. r1 ← r1 × r2
6. r1 ← r1 × r2
7. r1 ← r1 × r2
8. r1 ← r1 × r2
```

共执行8条指令

控制、寻址

- ▶ 降低控制开销：循环展开 (Loop unrolling)
- ▶ 降低寻址开销：强度削减 (Strength reduction)

1. $r3 \leftarrow 0$

2. $r2 \leftarrow [r3 \times 4 + 8000]$

3. $r1 \leftarrow r1 + r2$

4. $r3 \leftarrow r3 + 1$

5. 跳转至 2., 如果 $r3 < 8$

1. $r3 \leftarrow 8000$

2. $r2 \leftarrow [r3]$

3. $r1 \leftarrow r1 + r2$

4. $r3 \leftarrow r3 + 4$

5. 跳转至 2., 如果 $r3 < 8032$

通过线性变换减少寻址中使用的乘法和加法运算

控制、寻址

- ▶ 降低控制开销：循环展开 (Loop unrolling)
- ▶ 降低寻址开销：强度削减 (Strength reduction)
- ▶ 均为通用技巧，现代编译器已经尽力而为。
- ▶ 没有非常有效的优化方法！
 - ▶ 为什么？

控制、寻址

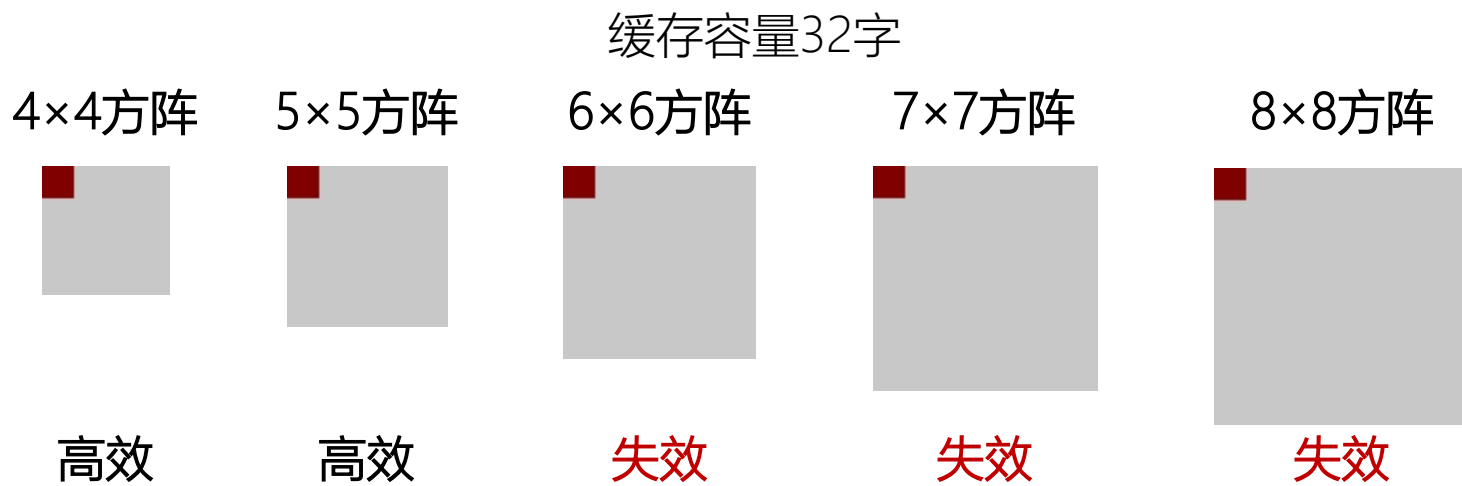
- ▶ 降低控制开销：循环展开 (Loop unrolling)
- ▶ 降低寻址开销：强度削减 (Strength reduction)
- ▶ 均为通用技巧，现代编译器已经尽力而为。
- ▶ 没有非常有效的优化方法！
 - ▶ 通用处理器为通用性而设计，深度学习只是其中一种应用；
 - ▶ 虽然深度学习程序行为规整，仍需较多指令才能定义清晰。

控制、寻址

- ▶ 降低控制开销：循环展开 (Loop unrolling)
- ▶ 降低寻址开销：强度削减 (Strength reduction)
- ▶ 均为通用技巧，现代编译器已经尽力而为。
- ▶ 没有非常有效的优化方法！
 - ▶ 通用处理器为通用性而设计，深度学习只是其中一种应用；
 - ▶ 虽然深度学习程序行为规整，仍需较多指令才能定义清晰。
 - ▶ 每访问一个数据，都必须计算其地址，并控制循环条件！

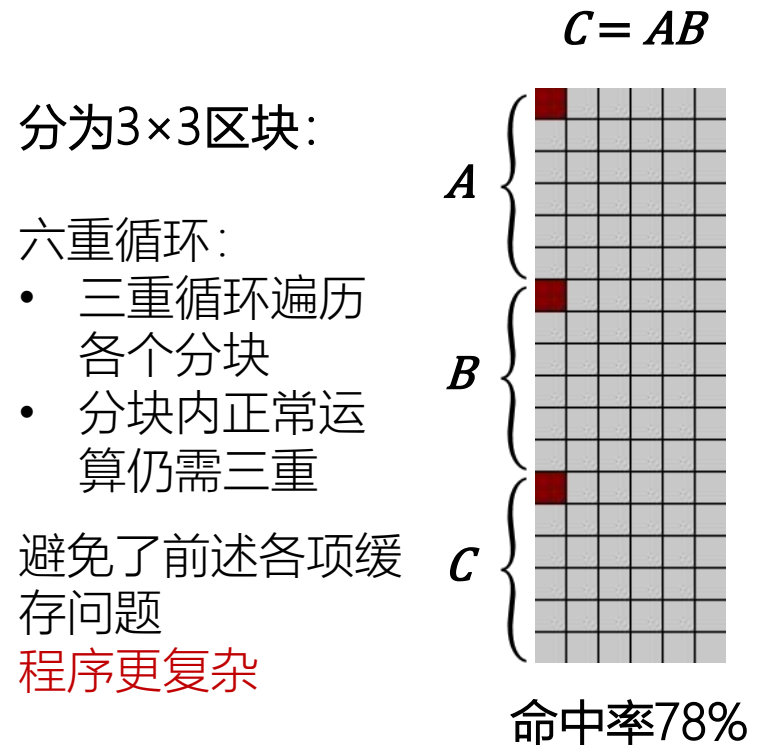
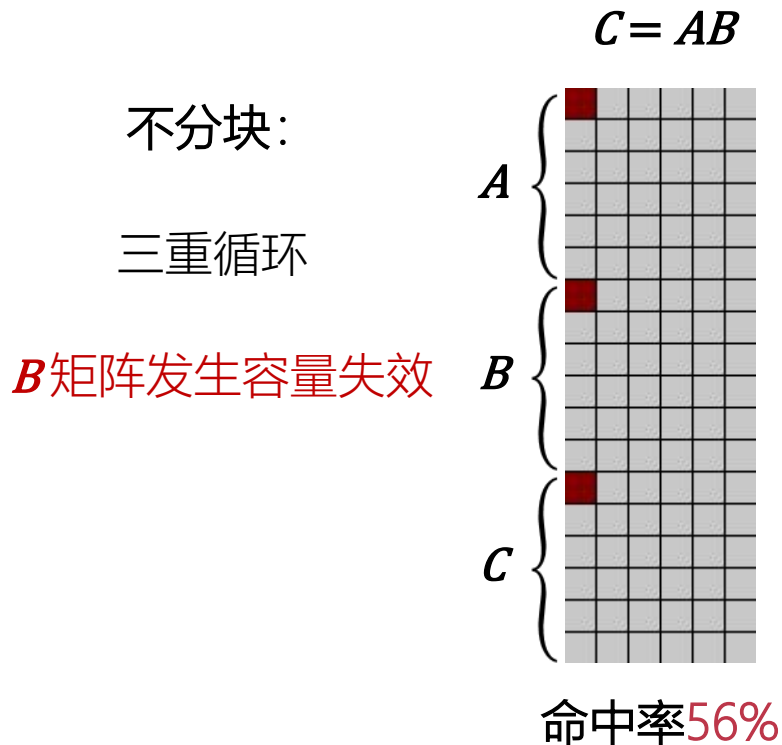
访存

- ▶ 除了控制和寻址开销之外，还有访存开销！
- ▶ 容量失效：循环访问数据超过缓存容量，LRU缓存失效
(LRU：最长时间未使用的缓存替换策略)



访存优化

- 分块 (tiling) : 将运算分解至固定尺寸的区域处理



访存优化

► 通用处理器上实现矩阵乘法，有两种实现方式：

► 递归（分治法）

if $N > 1$:

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \cdot \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

$$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21}$$

$$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22}$$

$$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21}$$

$$C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22}$$

else:

return $A*B$

► 迭代（三重循环）

for $i < N$:

for $j < N$:

for $k < N$:

$C[i][j] += A[i][k]*B[k][j]$

谁更高效？

访存优化

► 通用处理器上实现矩乘，这两种实现方式：

► 递归（分治法）

if $N > 1$:

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \cdot \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

$$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21}$$

$$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22}$$

$$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21}$$

$$C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22}$$

else:

return $A * B$

► 迭代（三重循环）

for $i < N$:

for $j < N$:

for $k < N$:

$C[i][j] += A[i][k] * B[k][j]$

谁更高效？ **没有定论！** 需要通过测试来确认。

递归优势：

- 相当于实现了不同尺寸的分块，缓存友好；
- 可以采用Strassen等快速算法！

迭代优势：

- 程序简单；
- 控制、寻址开销较低。

讨论

- ▶ 通用处理器独特的优势：普及、廉价、灵活
- ▶ 缺陷：较高的控制、寻址开销；复杂的访存优化策略。
 - ▶ 控制、寻址开销仅凭软件优化难以改善。
- ▶ 如何从架构上改善？

讨论

- ▶ 通用处理器独特的优势：普及、廉价、灵活
- ▶ 缺陷：较高的控制、寻址开销；复杂的访存优化策略。
 - ▶ 控制、寻址开销仅凭软件优化难以改善。
- ▶ 如何从架构上改善？
 - ▶ 观察：深度学习处理数据以向量、矩阵为主，数量较多。

讨论

- ▶ 通用处理器独特的优势：普及、廉价、灵活
- ▶ 缺陷：较高的控制、寻址开销；复杂的访存优化策略。
 - ▶ 控制、寻址开销仅凭软件优化难以改善。
- ▶ 如何从架构上改善？
 - ▶ 观察：深度学习处理数据以向量、矩阵为主，数量较多。
 - ▶ 思路：允许一条指令并行操作多个数据？

讨论

- ▶ 通用处理器独特的优势：普及、廉价、灵活
- ▶ 缺陷：较高的控制、寻址开销；复杂的访存优化策略。
 - ▶ 控制、寻址开销仅凭软件优化难以改善。
- ▶ 如何从架构上改善？
 - ▶ 观察：深度学习处理数据以向量、矩阵为主，数量较多。
 - ▶ 思路：允许一条指令并行操作多个数据？
 - ▶ 预期效果：摊薄控制和寻址开销！